COMP2271: Computer Graphics

Part 1: Introduction to Graphics & The Rendering Pipeline

Dr. Frederick Li

Department of Computer Science
Durham University

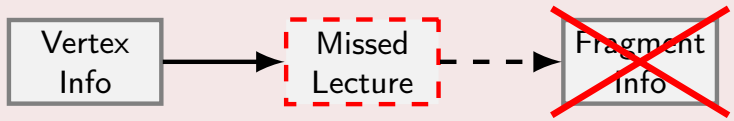
How to Succeed in Studying Computer Graphics

1. **The "Golden Triangle" of Resources**
   - **Lecture Slides (The Skeleton):**
     - "Set in stone" core info.
     - Defines the exact scope of the exam.
   - **Handouts (The Muscle):**
     - Explains *t*heories and methods.
     - Contains detailed code with explanations that won't fit on slides.
   - **Live Lectures (The Pulse):**
     - Live code walkthroughs.
     - Demos.

Warning: The "Pipeline" Effect

Do Not Skip Lectures. This module works like the **Rendering Pipeline**:

- It is a sequential process.
- If you miss *Vertex Processing*, *Fragment Processing* will not make sense.



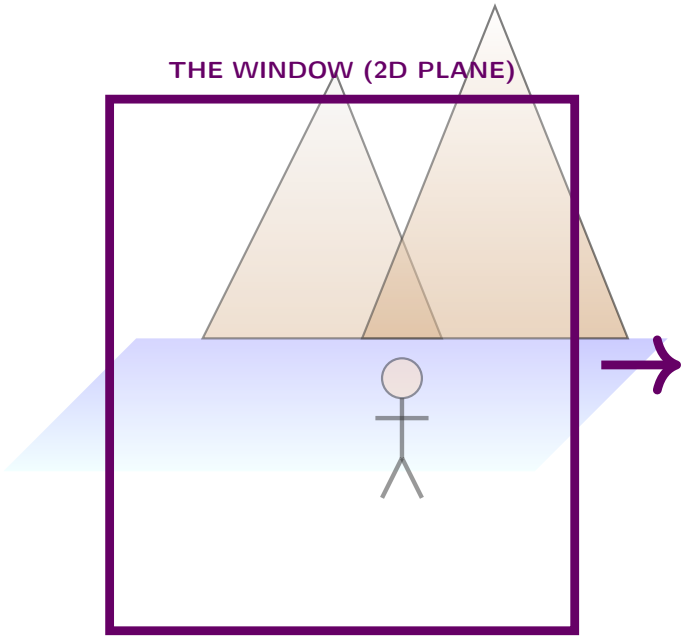
- **Don't Just Memorise:** Understand the data flow.

The Hidden World Behind the Glass

A Thought Experiment

Imagine for a moment that your screen is not a flat surface, but a **window**.

- You look through it and see a vast, 3D world with **mountains, oceans, and characters**. It feels real.



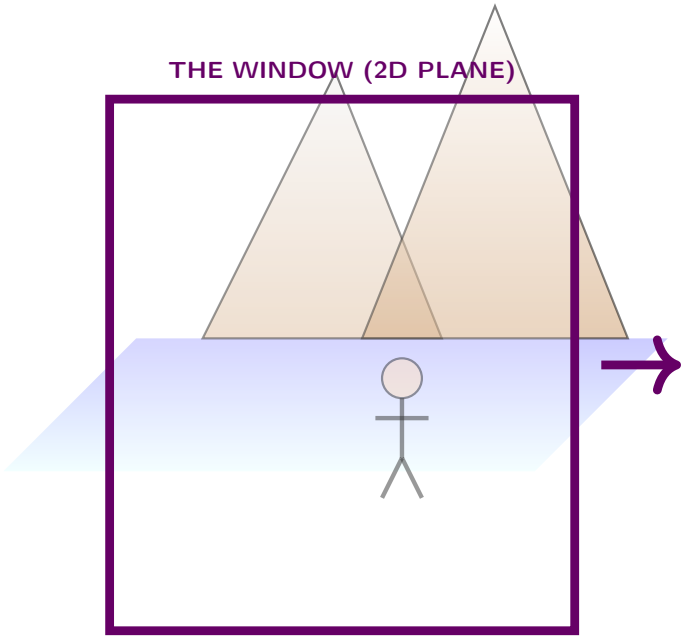
Math $\rightarrow$ *GPU* $\rightarrow$ *Deception*

The Hidden World Behind the Glass

A Thought Experiment

Imagine for a moment that your screen is not a flat surface, but a **window**.

- You look through it and see a vast, 3D world with **mountains, oceans, and characters**. It feels real.
- But here is the truth: **None of it exists**.



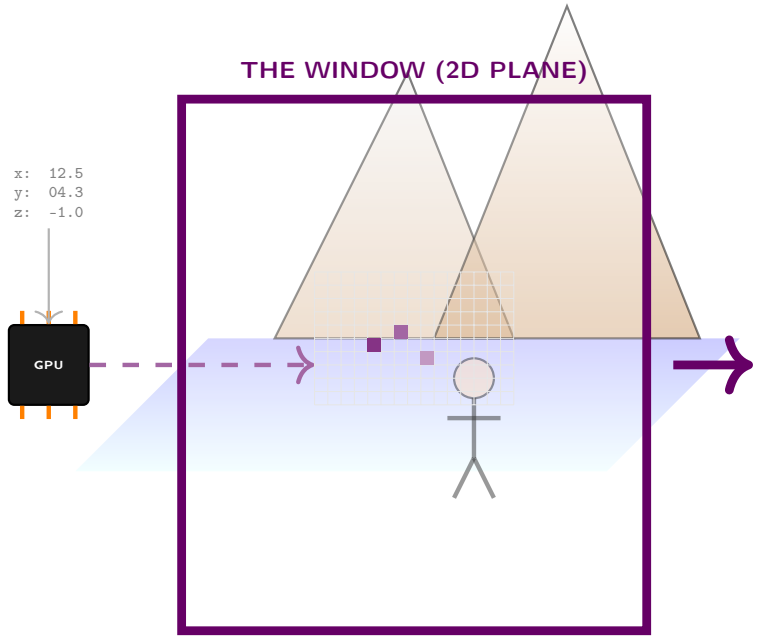
Math $\rightarrow$ *GPU* $\rightarrow$ *Deception*

The Hidden World Behind the Glass

A Thought Experiment

Imagine for a moment that your screen is not a flat surface, but a **window**.

- You look through it and see a vast, 3D world with **mountains, oceans, and characters**. It feels real.
- But here is the truth: **None of it exists**.
- Every millisecond, your computer performs a magic trick. It takes a list of **boring numbers (coordinates)** and furiously paints millions of **tiny coloured dots** to *trick* your brain into seeing depth.



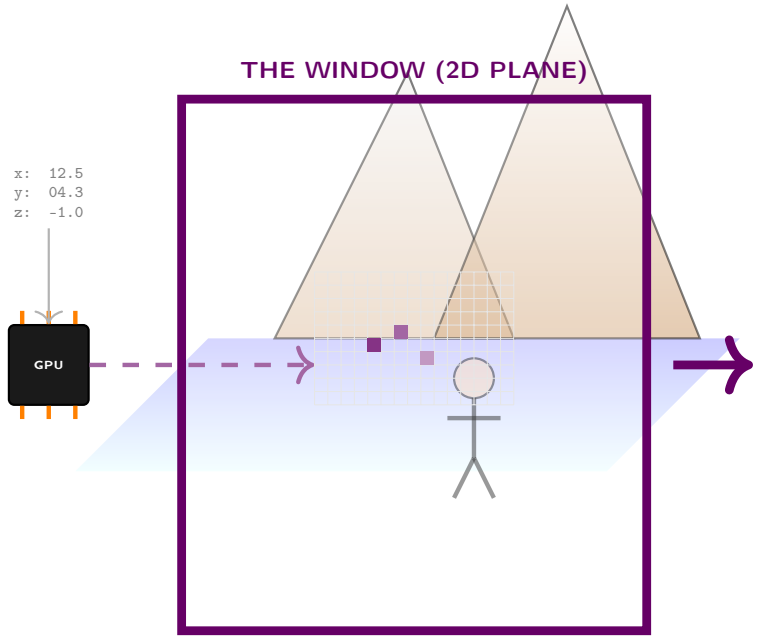
Math $\rightarrow$ *GPU* $\rightarrow$ *Deception*

The Hidden World Behind the Glass

A Thought Experiment

Imagine for a moment that your screen is not a flat surface, but a **window**.

- You look through it and see a vast, 3D world with **mountains, oceans, and characters**. It feels real.
- But here is the truth: **None of it exists**.
- Every millisecond, your computer performs a magic trick. It takes a list of **boring numbers (coordinates)** and furiously paints millions of **tiny coloured dots** to *trick* your brain into seeing depth.



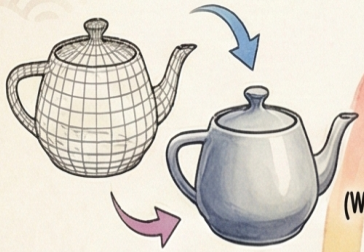
Math $\rightarrow$ *GPU* $\rightarrow$ *Deception*

The Question

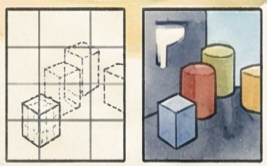
How do we teach a **rock (the silicon chip)** to perform this artistic deception 60 times a second? That is the story of this module.

50 Years of Computer Graphics: A Watercolor History Scroll & Key Milestones

[1970s: Foundations of Rasterization



1975: Utah Teapot & Shading Models (Phong, Gouraud). The standard reference object.



1974: Z-Buffer Algorithm. Solving hidden surfaces.

[1980s: Ray Tracing & Standards

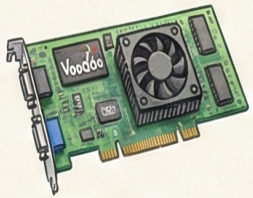
1908: Recursive Ray Tracing (Whitted). Global Illumination begins.

1987: VGA Standard. Baseline for PC graphics.

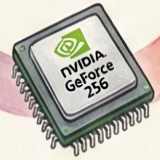
1988: RenderMan Interface. Film rendering standard.

[1990s: 3D Acceleration Revolution

1992/95: OpenGL & DirectX APIs. Cross-platform & gaming standards.

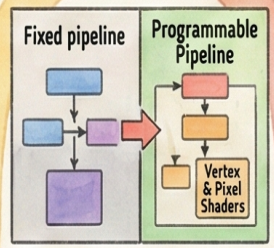


1996: 3dfx Voodoo. 3D acceleration for gamers.

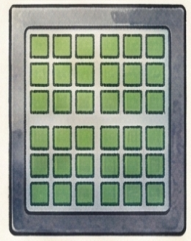


1999: The First "GPU" (GeForce 256). Hardware T&L offloading.

[2000s: Programmability & Unified Shaders



2001: Programmable Shaders (GeForce 3). Custom effects.

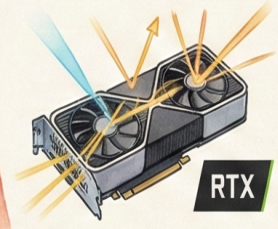


Unified Shader Arch

2006: Unified Architecture. Efficient parallel processing.

[2010s: PBR & Real-Time Ray Tracing

2012+: Physically Based Rendering (PBR). Standardized realism.



2018: Real-Time Ray Tracing (RTX). Hybrid rendering viable.

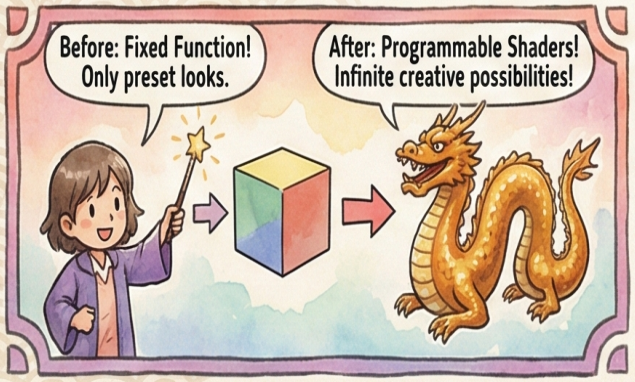
2016: Vulkan API. Low-level, high-efficiency access.

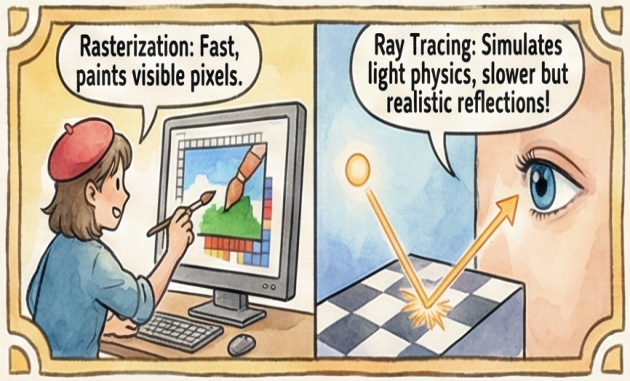
[2020s: AI & Neural Graphics

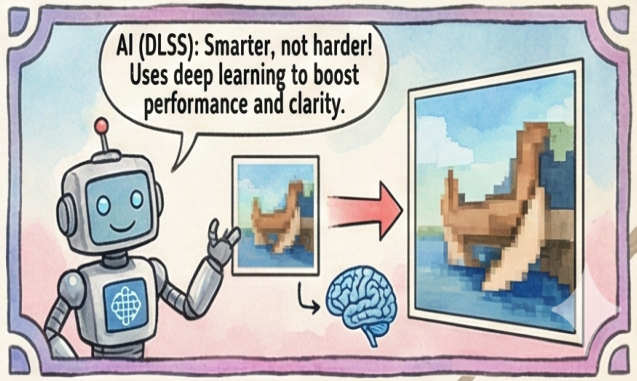
2020: AI Upscaling (DLSS). High res from low input.

NeRF / Neural Rendering

2022+: Neural Rendering. AI reconstructs 3D scenes.







Outline

Graphics in the Data Science Pipeline

More than just Charts

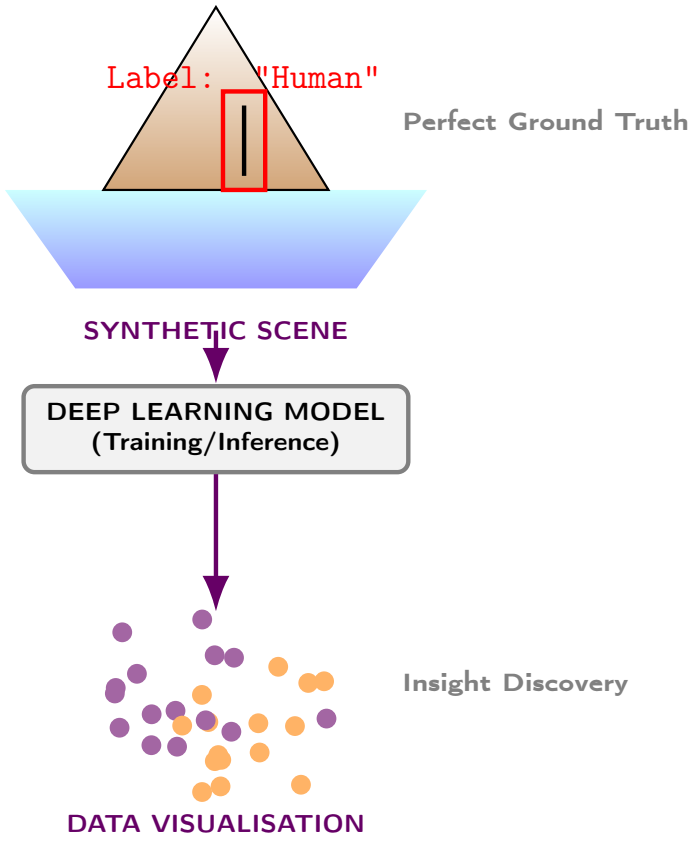
Computer Graphics (CG) is not just for games; it is a critical component of the modern Data Science and AI pipeline. It serves as an **input generator** and an **output visualiser**.

Input: Synthetic Data Generation

CG generates vast amounts of labelled training data for Deep Learning models when real-world data is scarce, expensive, or dangerous.

Output: Advanced Visualisation

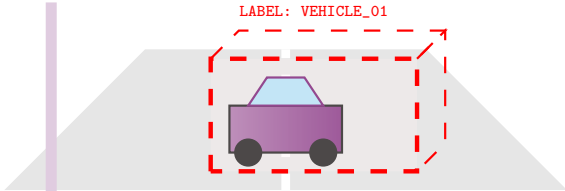
CG provides the tools to interpret complex model outputs, rendering high-dimensional data into human-understandable visual formats.



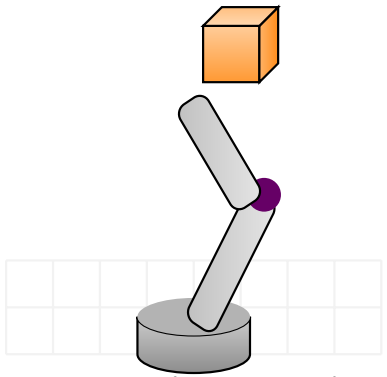
CG as Input for Deep Learning

Training AI with Virtual Worlds

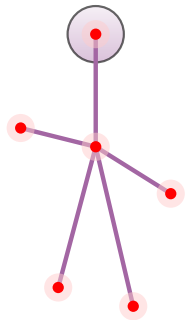
- **Autonomous Driving:**
  - *Problem:* Collecting real-world crash data is dangerous.
  - *CG Solution:* Render photorealistic virtual cities. We know the exact position of every car and pedestrian (perfect "Ground Truth" labels) to train object detection networks (e.g., YOLO, R-CNN).
- **Robotics & Simulation:**
  - *Problem:* Robots need millions of trials to learn to grasp objects.
  - *CG Solution:* Use physics-based rendering to simulate robot arms interacting with 3D objects in a virtual environment (Sim-to-Real transfer).
- **Pose Estimation:**
  - *CG Solution:* Render 3D human models in thousands of poses to train networks to recognise human body language from 2D video.



Autonomous Driving



Robotics (Sim-to-Real)

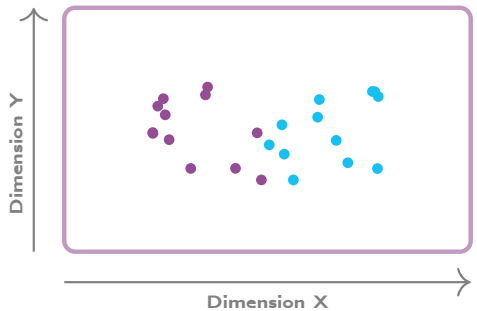


Pose Estimation

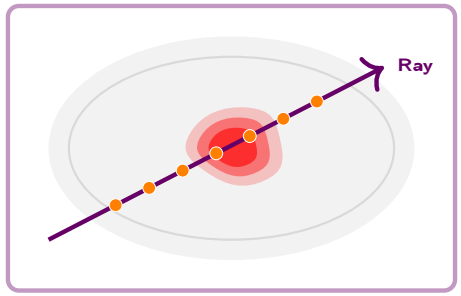
CG for Output Visualisation

Making Sense of the "Black Box"

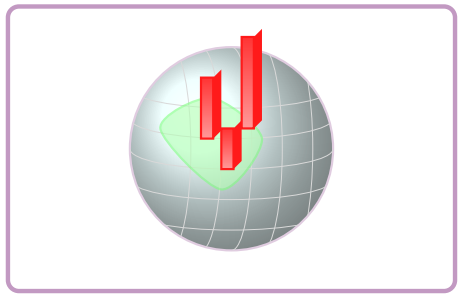
- **Dimensionality Reduction (t-SNE / UMAP):**
  - Transforming high-dimensional feature vectors into 3D scatter plots. WebGL allows analysts to fly through millions of data points to find clusters and outliers in real-time.
- **Volume Rendering (Medical AI):**
  - Deep learning models segment tumours in MRI scans. CG techniques (Ray Marching) visualise these segmented 3D volumes, allowing doctors to see the exact shape and location of the pathology.
- **Geospatial Digital Twins:**
  - Visualising AI predictions for climate change or traffic flow on a 3D globe, using texture mapping and height maps to represent data density.



Cluster Interpretation



Ray Marching (3D Pathology Segmentation)


Geospatial Digital Twin (AI Height Maps)

What is Computer Graphics?

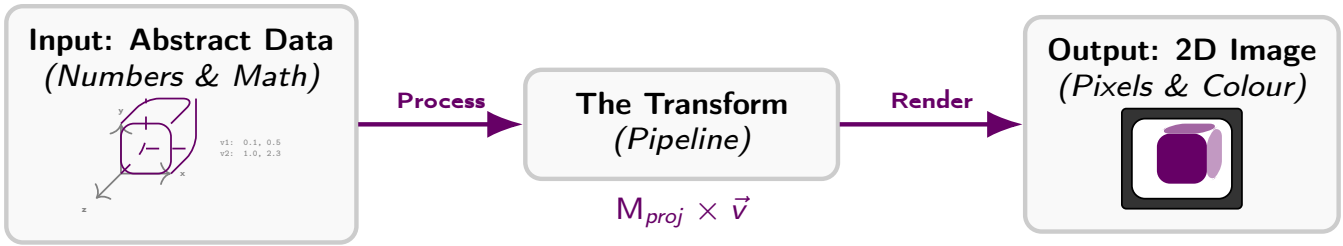
The Core Question

Computer Graphics is the field dedicated to generating, manipulating, and synthesising visual content with computers.

The primary goal is to transform an **abstract description** of a scene (shapes, materials, lights, camera) into a **compelling 2D image**.

It is a powerful blend of multiple disciplines:

- **Physics:** Simulating light transport for realism.
- **Mathematics:** Using linear algebra for transformations.
- **Computational Science:** Developing efficient algorithms.
- **Art:** Applying principles of composition and colour.



The Three Pillars of Graphics

An Interconnected Relationship

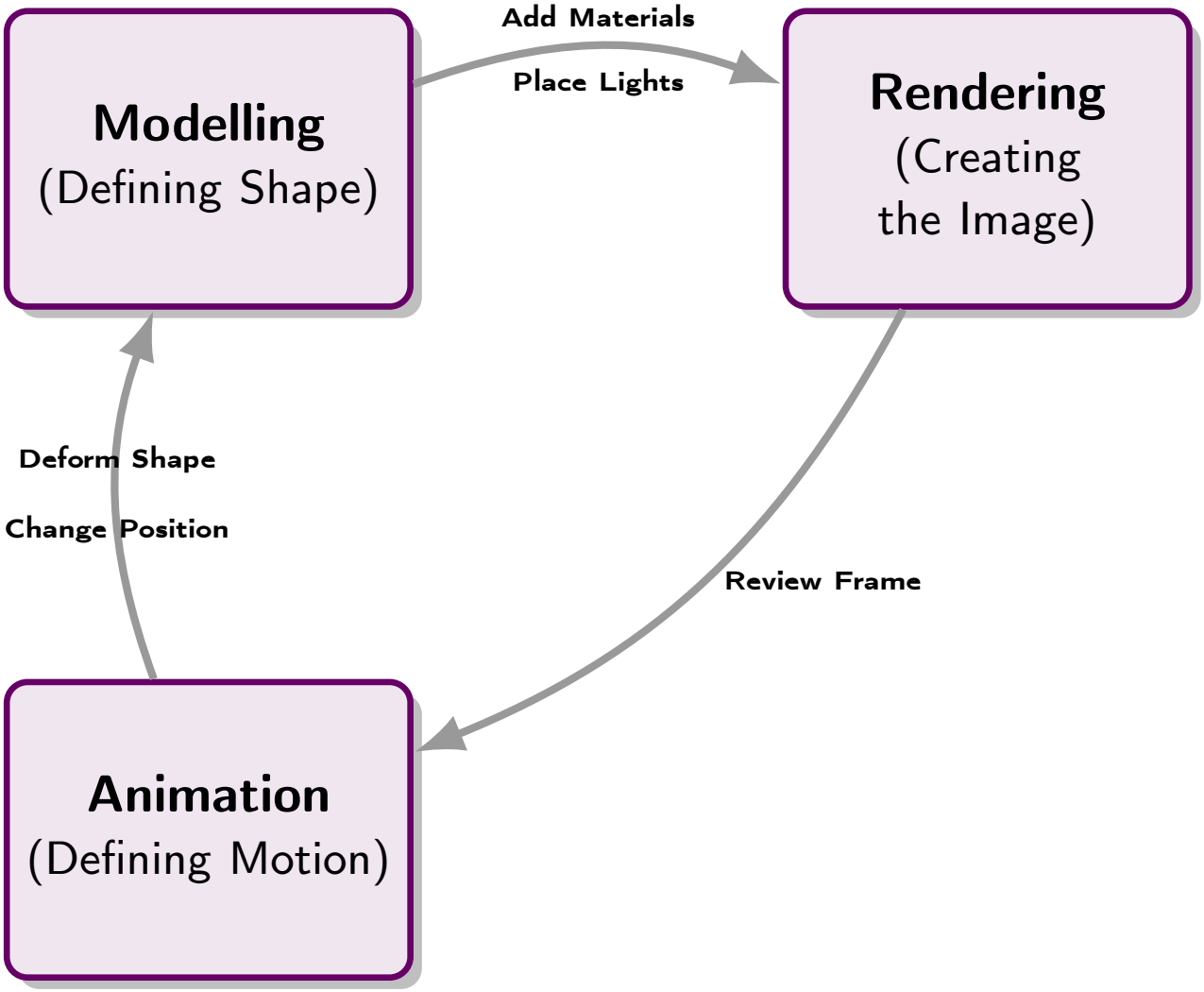


Figure: Modelling, Rendering, and Animation form a cyclical process.

Rendering Philosophies Compared

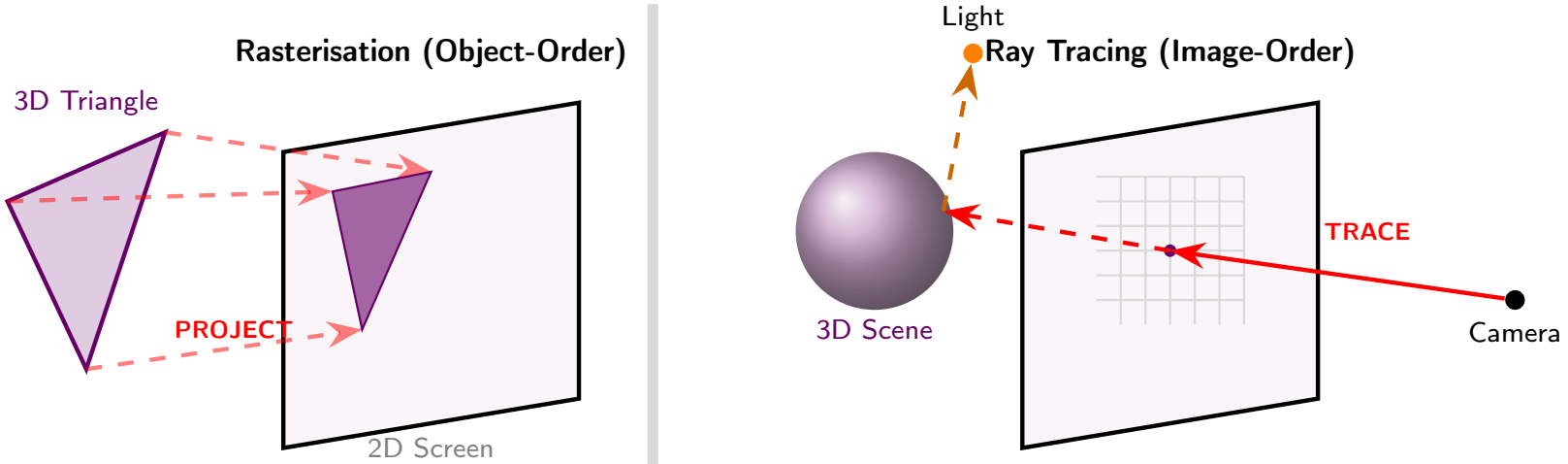


Figure: Rasterisation projects objects; Ray Tracing casts rays from the camera.

Course Focus

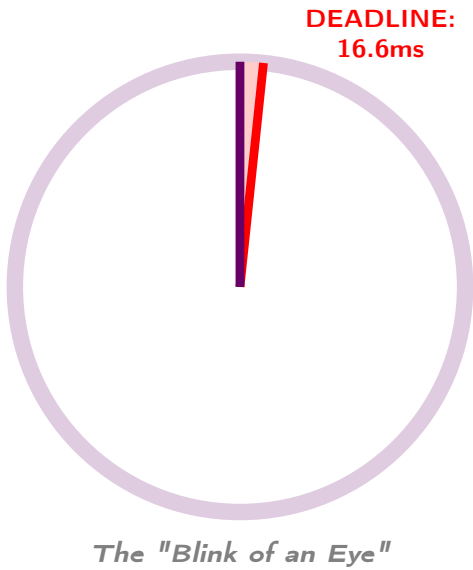
For this course, we will focus on Rasterisation, namely the object-order process that acts as the bridge between continuous mathematics and discrete pixels by projecting 3D triangles onto the 2D screen and "filling in" the pixels they cover, which is the core of real-time graphics APIs like WebGL.

The 16-Millisecond Miracle

A Race Against Time

We have talked about algorithms, but let's talk about **speed**. To run a game at 60 FPS, you have exactly **16.6 milliseconds** to draw one frame.

In that blink of an eye, the GPU must:



The "Blink of an Eye"

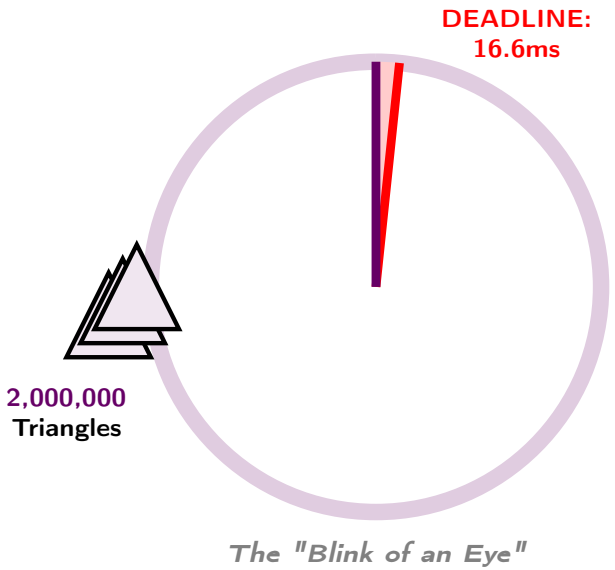
The 16-Millisecond Miracle

A Race Against Time

We have talked about algorithms, but let's talk about **speed**. To run a game at 60 FPS, you have exactly **16.6 milliseconds** to draw one frame.

In that blink of an eye, the GPU must:

1. Take **2 million** triangles from a city model.



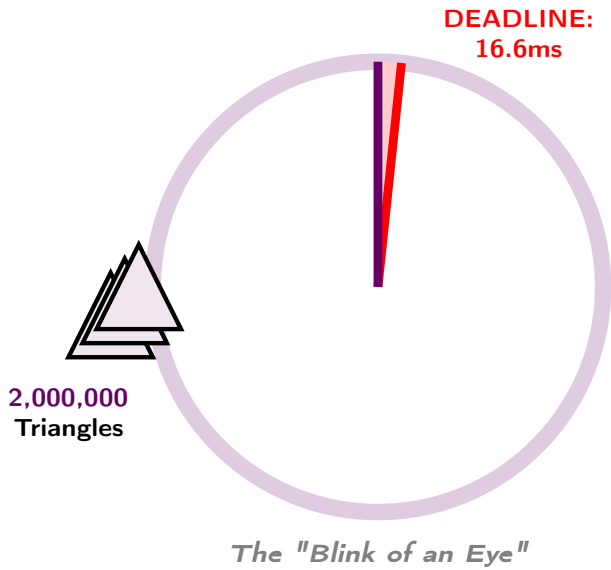
The 16-Millisecond Miracle

A Race Against Time

We have talked about algorithms, but let's talk about **speed**. To run a game at 60 FPS, you have exactly **16.6 milliseconds** to draw one frame.

In that blink of an eye, the GPU must:

1. Take **2 million** triangles from a city model.
2. Transform every vertex (x, y, z) into screen space.



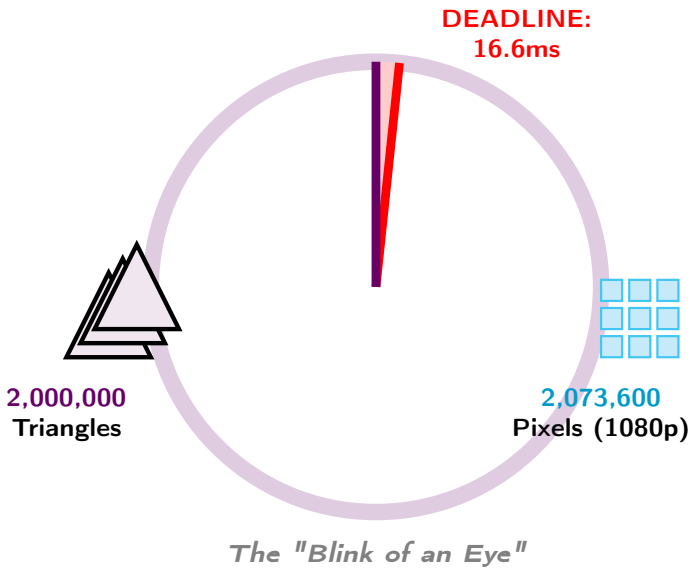
The 16-Millisecond Miracle

A Race Against Time

We have talked about algorithms, but let's talk about **speed**. To run a game at 60 FPS, you have exactly **16.6 milliseconds** to draw one frame.

In that blink of an eye, the GPU must:

1. Take **2 million** triangles from a city model.
2. Transform every vertex (x, y, z) into screen space.
3. Calculate lighting for **2 million** pixels (1080p).



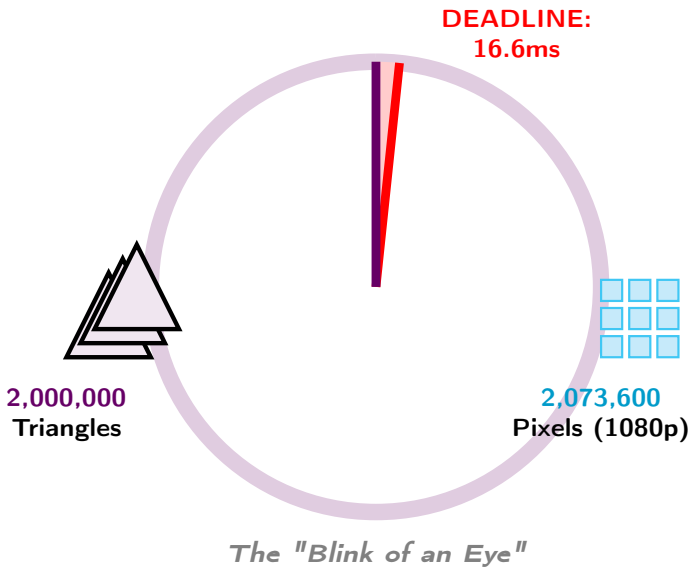
The 16-Millisecond Miracle

A Race Against Time

We have talked about algorithms, but let's talk about **speed**. To run a game at 60 FPS, you have exactly **16.6 milliseconds** to draw one frame.

In that blink of an eye, the GPU must:

1. Take **2 million** triangles from a city model.
2. Transform every vertex (x, y, z) into screen space.
3. Calculate lighting for **2 million** pixels (1080p).
4. Figure out which pixels are hidden behind others.



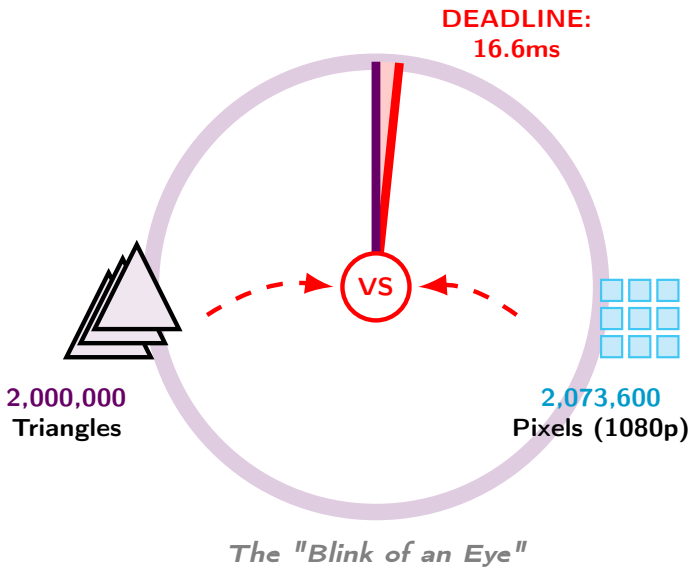
The 16-Millisecond Miracle

A Race Against Time

We have talked about algorithms, but let's talk about **speed**. To run a game at 60 FPS, you have exactly **16.6 milliseconds** to draw one frame.

In that blink of an eye, the GPU must:

1. Take **2 million** triangles from a city model.
2. Transform every vertex (x, y, z) into screen space.
3. Calculate lighting for **2 million** pixels (1080p).
4. Figure out which pixels are hidden behind others.



The Real Challenge

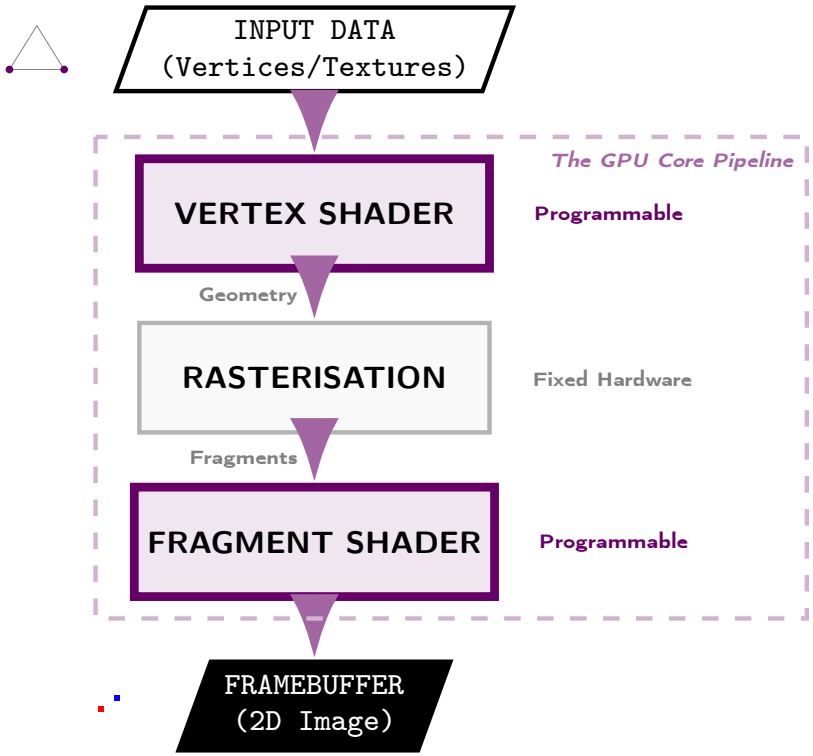
This is not just math. It is a **logistical nightmare**. Graphics programming is the art of organising data so efficiently that hardware doesn't choke. It is like a marathon where 2 million runners must cross the finish line in 0.016 seconds.

The Rendering Pipeline

A Conceptual Model (WebGL Perspective)

The **Rendering Pipeline** is the sequence of steps a GPU follows to convert a 3D scene into a 2D image.

- **Analogy:** A **factory assembly line**.
- **Input:** Raw materials like vertex data, textures, and camera settings.
- **Process:** Data moves through distinct stages, where some are **fixed-function**, others are **programmable** via shaders.
- **Output:** A finished 2D image stored in the **framebuffer**.



The "Assembly Line" Flow

(A WebGL Perspective)

The Role of the GPU

Modern GPUs are specialised, massively parallel processors engineered to execute this pipeline at incredible speeds.

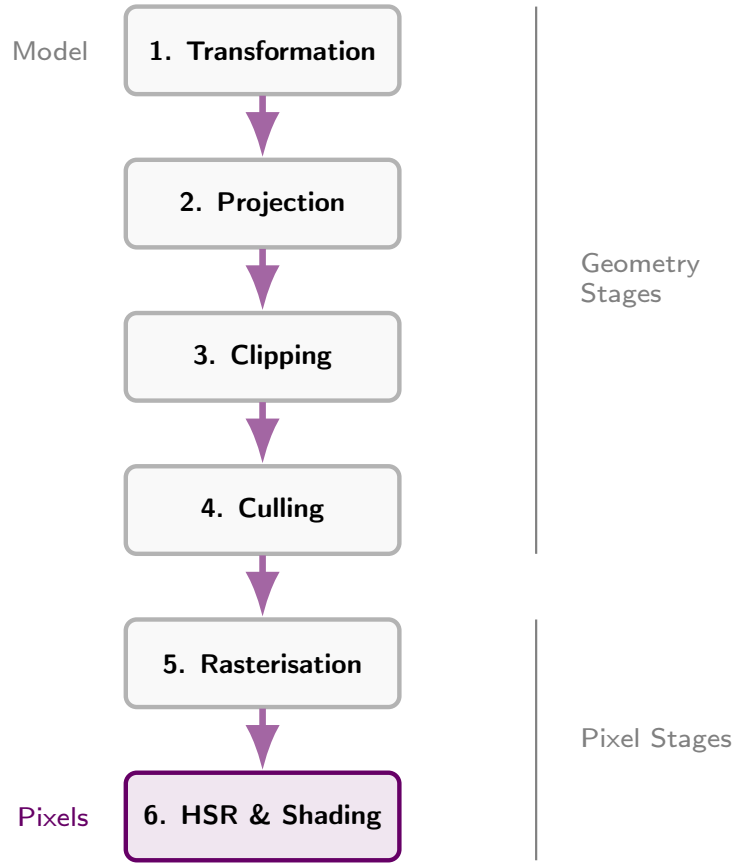
The Rendering Pipeline

A Deeper Look: The 6 Hardware Stages

These steps break down the *hardware-level* operations. Modern APIs group these, but understanding the hardware logic is crucial.

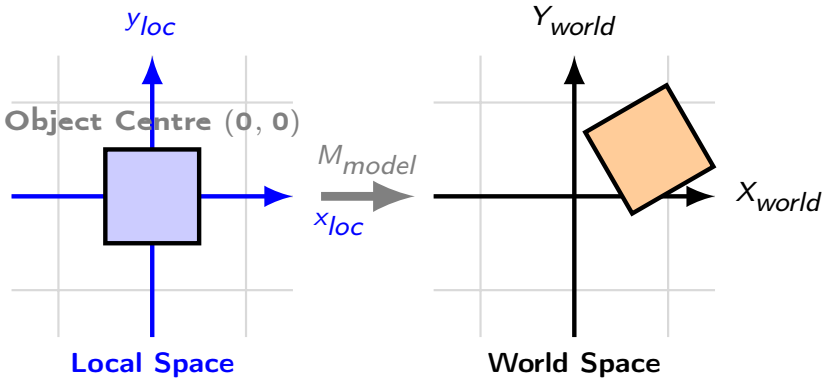
The Fixed-Function Flow

1. **Object Transformation**: Local space → World space.
2. **Perspective Projection**: 3D World → 2D Screen plane.
3. **Clipping**: Discarding geometry outside the view.
4. **Backface Culling**: Discarding rear-facing surfaces.
5. **Rasterisation**: Geometry → Fragments (Potential Pixels).
6. **Hidden Surface Removal (HSR) & Shading**: Depth testing and colour calculation.



Stage 1: Object Transformation

Positioning in the World: Local vs. World Space



1. **Local Space (The Blueprint):**
   - Vertices are relative to the **object's centre** (e.g., nose is at $z = 1$).
   - *Why?* Allows us to define a shape once.
2. **World Space (The Scene):**
   - Vertices are relative to the **world origin** $(0, 0, 0)$.
   - *Relation:* $P_{world} = \text{Matrix} \times P_{local}$
   - *Why?* Allows us to place multiple copies (instances) of the same blueprint in different spots.

Technical Operation

- Uses linear algebra (4x4 matrices).
- Operations: Translation, Rotation, Scaling.
- **Parallelism:** Every vertex is calculated independently.

Concern: Overhead

Transforming millions of vertices on the CPU is too slow.

Solution: SIMD

GPUs use **SIMD** (Single Instruction, Multiple Data) to transform thousands of vertices at once.

Stage 2: Perspective Projection

Simulating the Camera Lens

This stage mimics human vision, where distant objects appear smaller. It transforms 3D coordinates into **Clip Space**.

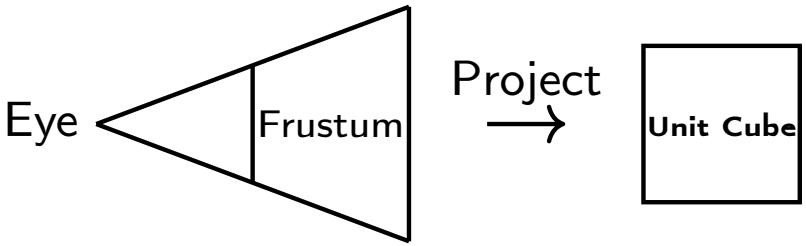
Technical Operation

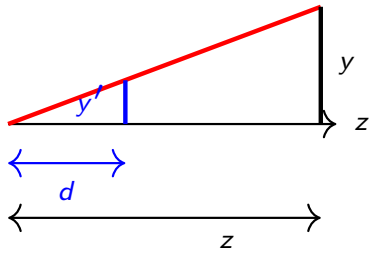
- Applies a projection matrix.
- Converts the viewing volume (view frustum) into a unit cube.
- **Perspective Divide:** Divides x, y, z by the homogeneous coordinate w (equivalent to depth z).

Concern: Depth Precision

Flattening 3D to 2D causes "Z-fighting"[a] if precision is low.

[a] Z-fighting: A rendering artifact where two primitives have similar distances to the camera, causing them to "flicker" as the GPU struggles to determine which is in front.





Similar Triangles:

$$\frac{y'}{d} = \frac{y}{z}$$

Result:

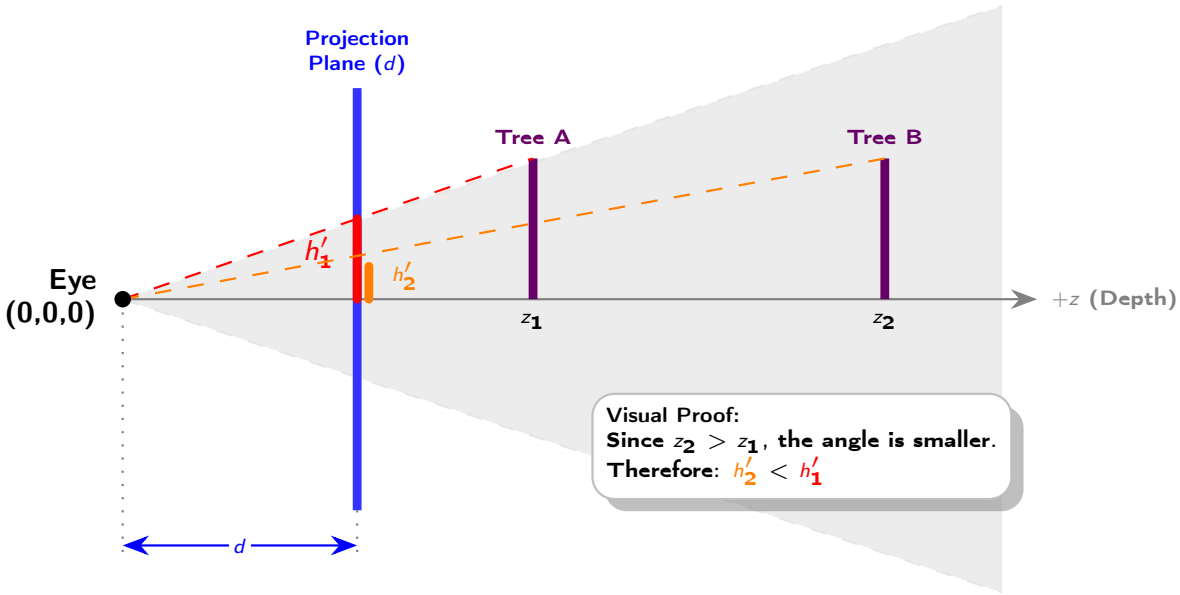
$$y' = d \cdot \frac{y}{z}$$

Why Do Distant Objects Look Smaller?

The Geometry of the "Perspective Divide"

The Logic of Perspective Projection:

- The Camera is positioned at the origin $(0, 0, 0)$.
- We trace a **visual ray** from the 3D object back to the eye.
- This ray intersects the 2D Projection Plane at distance d (focal length).
- Because the Object Triangle and Image Triangle share the same angle at the eye, they are **Similar Triangles**.



The Golden Rule

$$y' = \frac{d \cdot y}{z}$$

As depth (z) increases, the projected size (y') decreases.

This mathematical "shrinkage" provides the depth cue our brains interpret as distance.

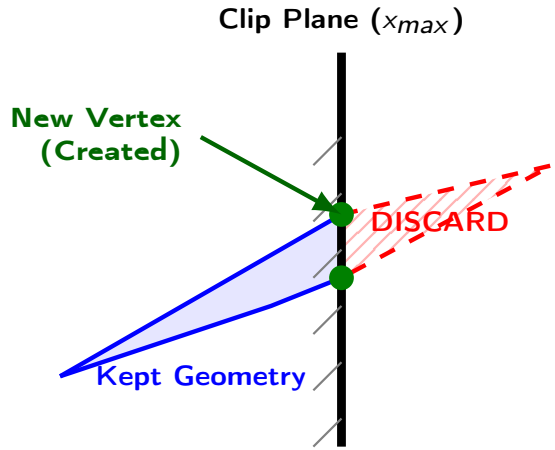
Stage 3: Clipping

Staying Within Bounds

Geometry outside the camera's view (view frustum) must be discarded before rasterisation to save processing power.

Technical Operation

The hardware checks if primitives lie within the defined viewing volume (frustum).



The Hardware Logic

Per Vertex Check

`if (x >` x_{max}`)`

TRUE → CLIP FALSE → KEEP

Main Concern: Partial Visibility

What happens if a triangle is half-in and half-out? We cannot simply discard it, nor can we draw the whole thing.

Solution: Retesselation

Algorithms (like Sutherland-Hodgman) mathematically **cut** the triangle at the edge, creating new vertices and triangles to fit inside.

Clipping Optimisation

Why We Clip After Projection

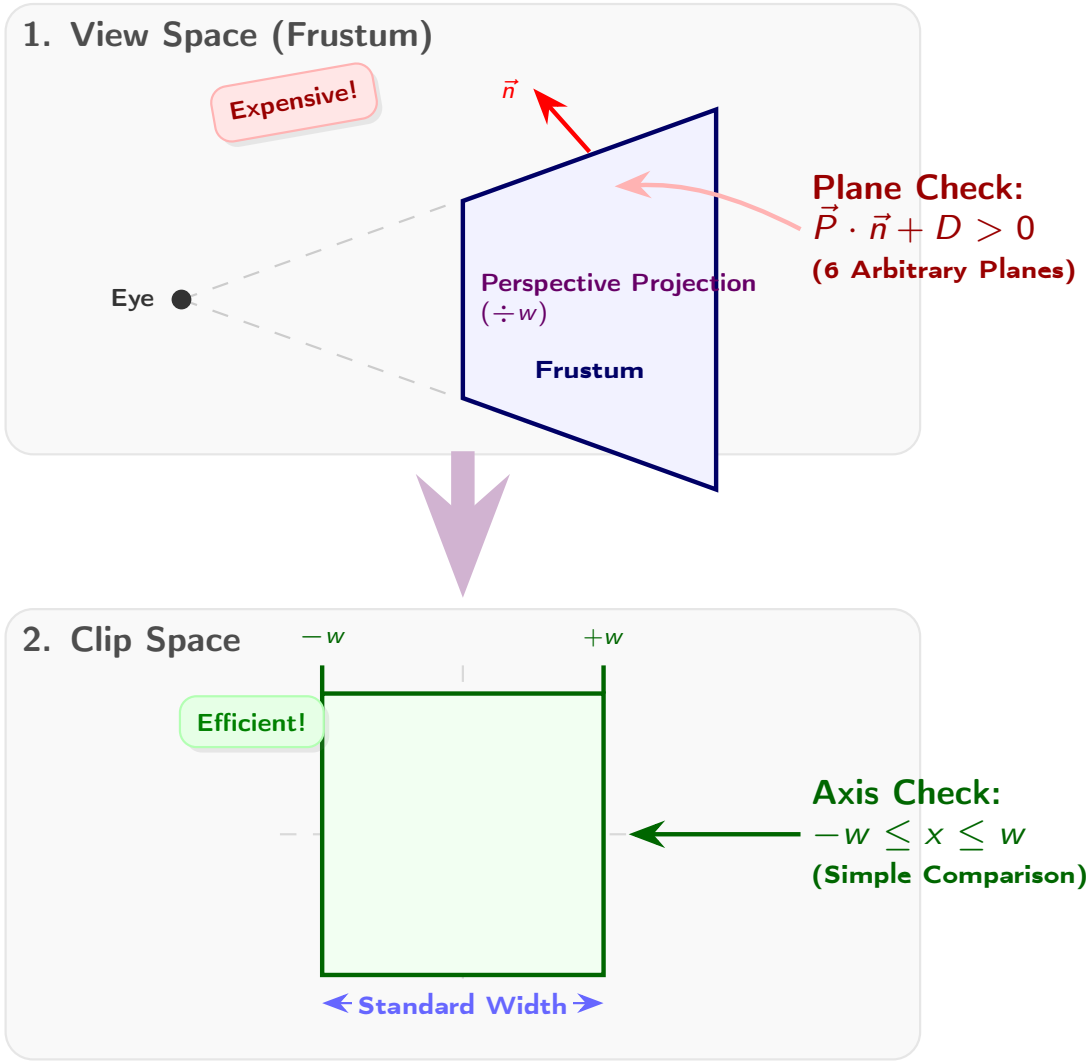
Why do we perform clipping **after** the projection matrix is applied?

1. **The Transformation** Perspective projection distorts space, turning the pyramid-shaped **Frustum** into a **Unit Cube** (Canonical View Volume).
2. **The Benefit**
   - **Before:** Checking if a point is inside a pyramid requires complex dot products with 6 arbitrary plane equations.
   - **After:** Checking if a point is inside a cube is a trivial axis-aligned check:

$$-w \leq x \leq w$$

Efficiency

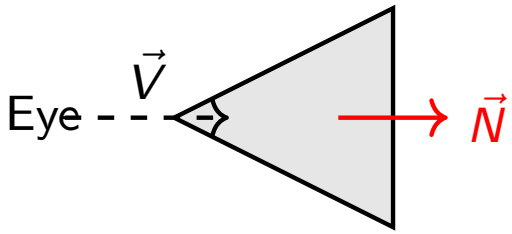
This transforms a heavy geometric problem into a lightweight inequality check.



Stage 4: Backface Culling

Discarding the Unseen

Most 3D objects are closed volumes. We never see the back/inside of a sphere or cube. Drawing these "back faces" is a waste.



Face pointing away

The Math Check:

$$\vec{V} \cdot \vec{N} > 0$$

$\Rightarrow$ **DISCARD**

Technical Operation

The hardware checks the **Winding Order** of vertices (Clockwise vs. Counter-Clockwise) in screen space.

Solution

If the dot product of the face normal and the camera view vector is > 0 (pointing away), the primitive is discarded immediately. This typically **halves** the rendering load.

Robust Culling in Clip Space

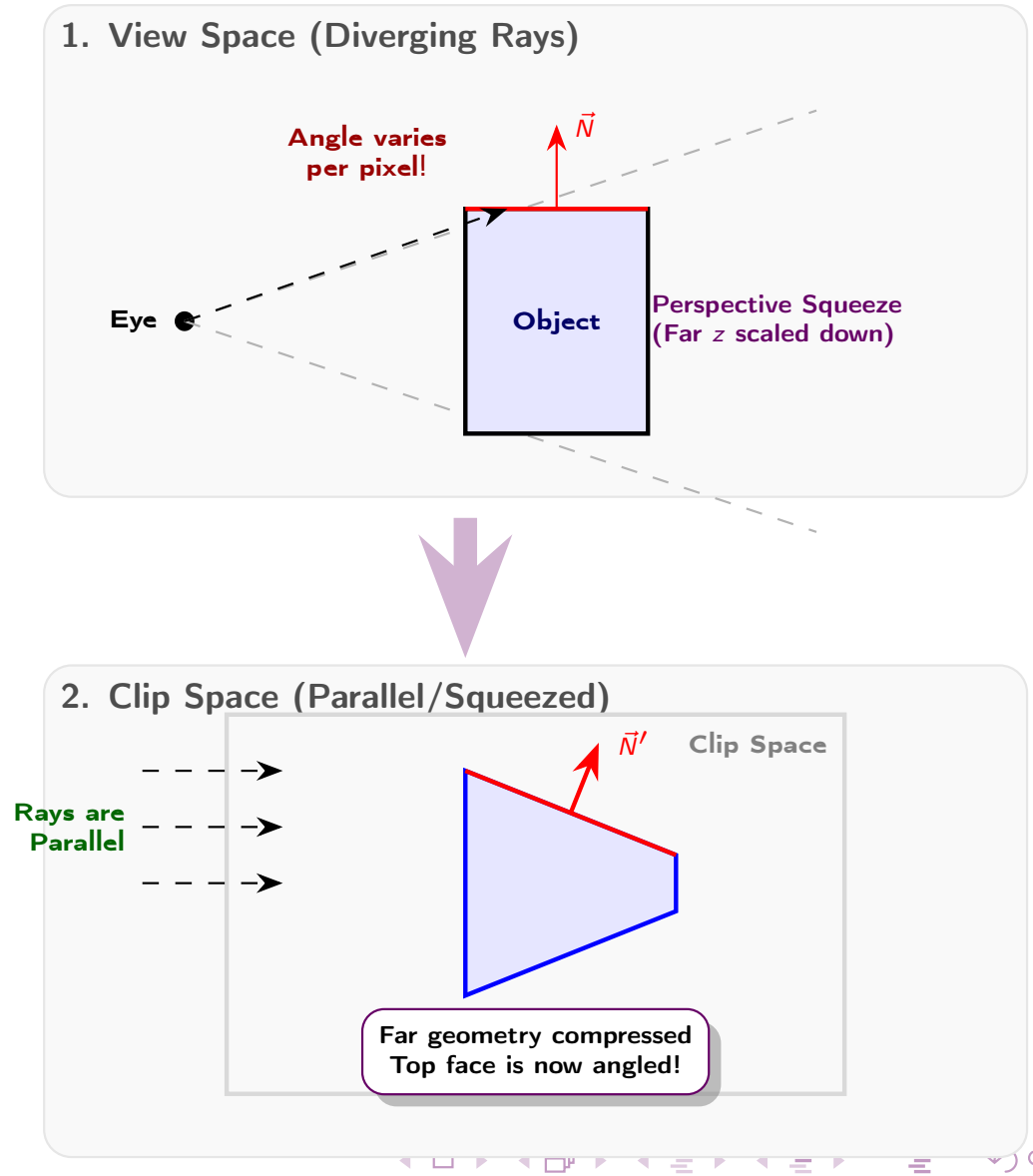
The "Squeeze" that Simplifies Visibility

The Perspective Squeeze

- In **View Space**, rays diverge. Checking visibility requires calculating angles relative to the specific eye position for every face.
- In **Clip Space**, the projection matrix **squeezes** the far end of the frustum into a cube.

Why is this Robust?

- This geometric distortion explicitly shrinks distant faces $(1/z)$.
- **Result:** View rays become **parallel**. The complex "am I looking at this?" 3D check becomes a simple 2D "is it clockwise?" check.



Stage 5: Rasterisation

The Bridge from Geometry to Pixels

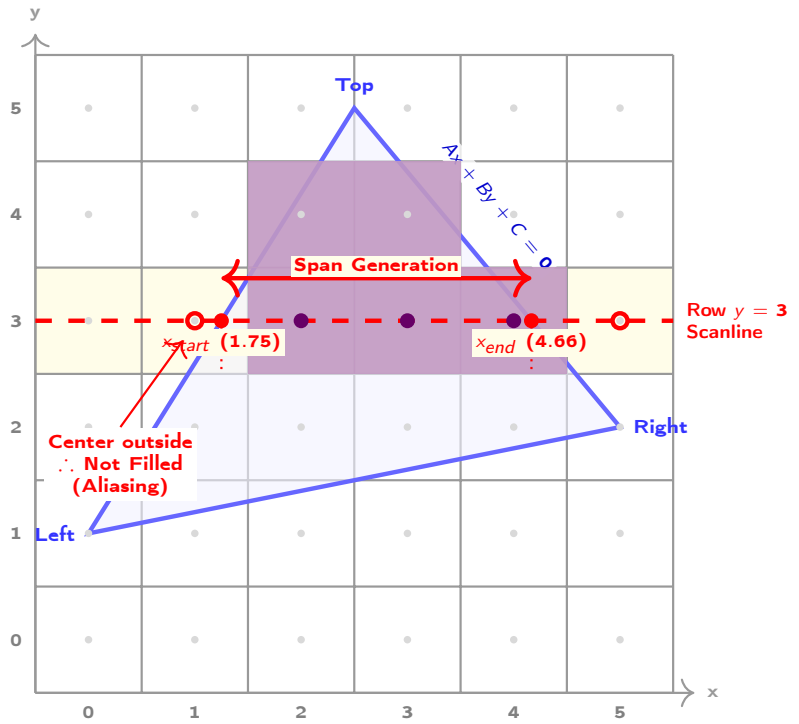
This stage converts continuous mathematical shapes into discrete screen pixels.

The Scanline Algorithm

1. **Setup:** Sort vertices by Y-coordinate to find the top, middle, and bottom.
2. **Walk Edges:** Calculate the equation for the left and right edges (slope).
3. **Scan & Span:** For every horizontal row (y):
   - Calculate x_{start} (Left Edge).
   - Calculate x_{end} (Right Edge).
   - **Emit Fragments** for all pixel centers inside this span.

The "Center Point" Rule

A pixel is generally colored only if its mathematically defined center lies inside the triangle edges.



Note: Attributes (Color, UVs) are **linearly interpolated** between x_{start} and x_{end} to calculate the final pixel fragment.

Stage 6: Hidden Surface Removal & Shading

Colour and Visibility

Finally, we calculate the colour and decide if the pixel is visible.

Shading

Calculates lighting based on normals, textures, and lights. (Handled by Fragment Shader) .

Why not the "Painter's Algorithm"?

Historically, we sorted polygons back-to-front.

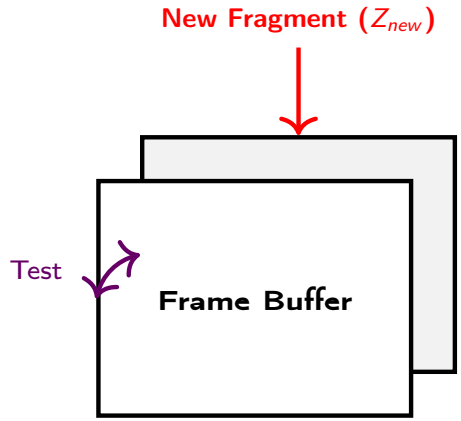
Why it failed:

- **Performance:** Sorting millions of triangles ($O(N \log N)$) is too slow.
- **Cyclic Overlaps:** No valid sort order exists!

Solution: The Z-Buffer

The hardware maintains a **Depth Buffer**.

- Stores depth (z) of closest object per pixel.
- If $Z_{new} < Z_{buffer}$, update; else, discard.



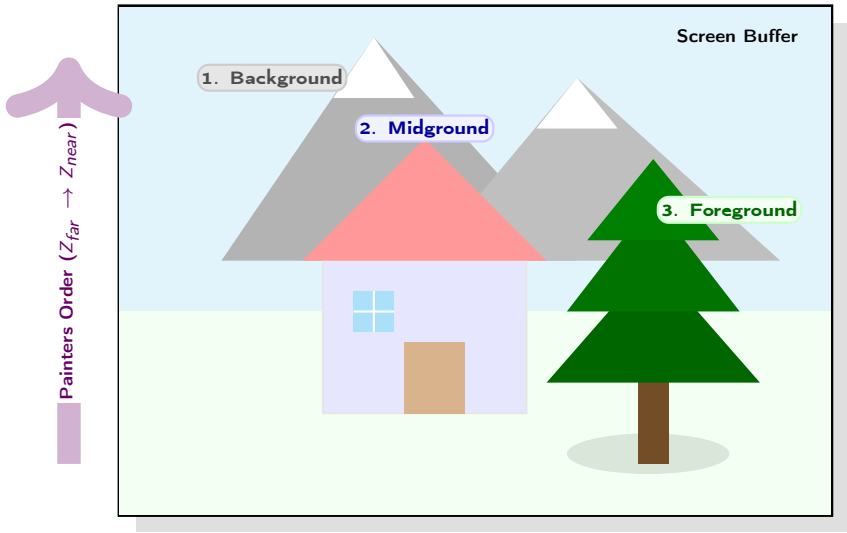
Hidden Surface Removal: The Painter's Algorithm

Object-Space Sorting (The Old Way)

Before modern Z-Buffers, graphics used an **Object-Space** approach inspired by physical painting.

The Algorithm:

1. **Sort:** Order all polygons by their depth (z) from farthest to nearest.
2. **Draw:** Render the farthest polygons first (Background).
3. **Overpaint:** Draw nearer polygons on top.



Why it feels intuitive

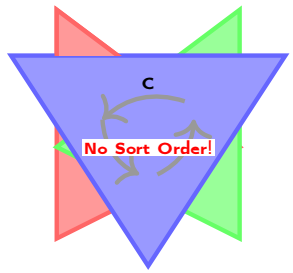
Just like an oil painter, you paint the sky, then the mountains, then the tree. The tree naturally covers the mountains.

The Fatal Flaw: Cyclic Overlap

If three triangles overlap in a cycle (A overlaps B, B overlaps C, C overlaps A), **no mathematical sort order exists**.

The Failure Case: Cyclic Overlap

("Rock Paper Scissors" Geometry)



Hidden Surface Removal: Z-Buffering

Pixel-Perfect Visibility (No Sorting Required)

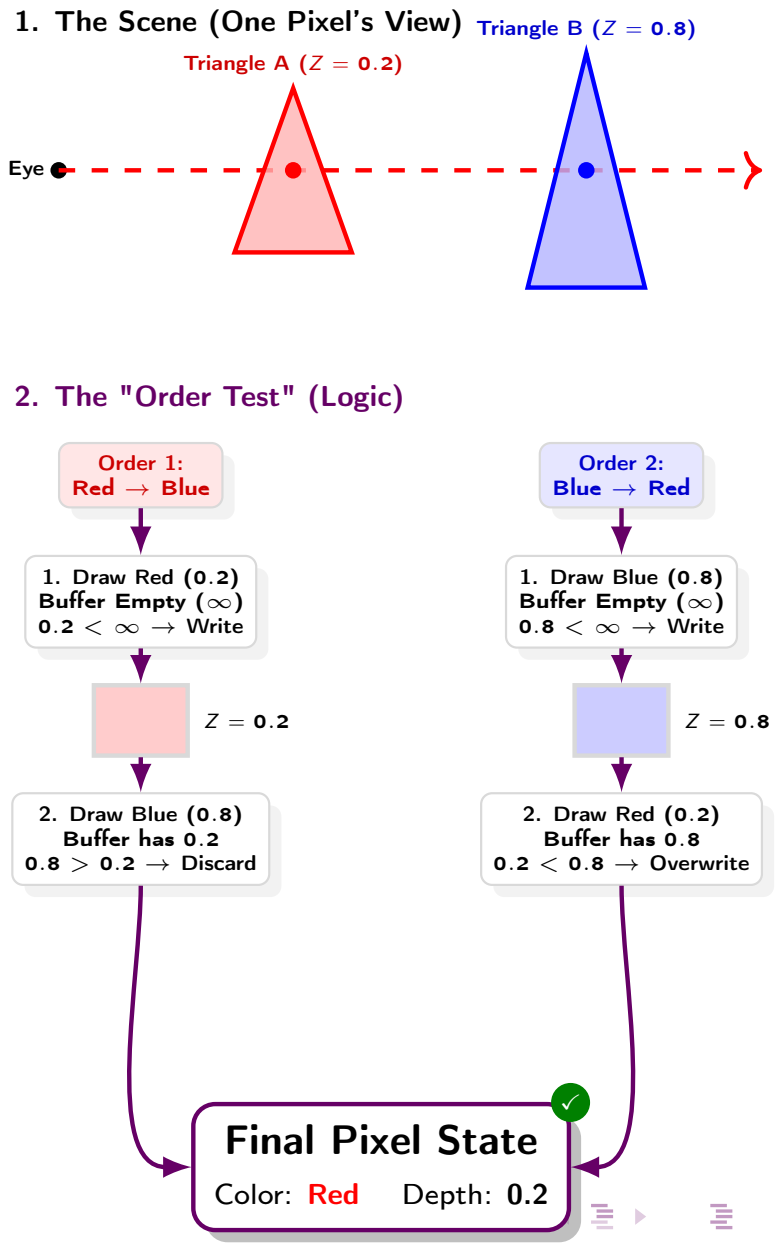
The **Z-Buffer** (Depth Buffer) solves visibility at the **pixel level**, fitting perfectly into the massive parallel architecture of modern GPUs.

The Mechanism:

1. **Memory:** The GPU allocates a 2D array of floats (Depth Buffer) matching the screen resolution.
2. **The Test:** For every fragment generated:
   - Check the stored depth (Z_{stored}) at that pixel (x, y).
   - **IF** $Z_{new} < Z_{stored}$:
     $\rightarrow$ **Pass.** Update buffer with Z_{new} and write pixel colour.
   - **ELSE**:
     $\rightarrow$ **Discard.** Fragment is occluded.

Why it superseded Painter's Algorithm

- Order Independent: **You can submit triangles in any random order (for opaque objects).**
- Solves Cycles: **Handles intersecting geometry perfectly.**
- Fast: $O(1)$ **constant time check per pixel.**

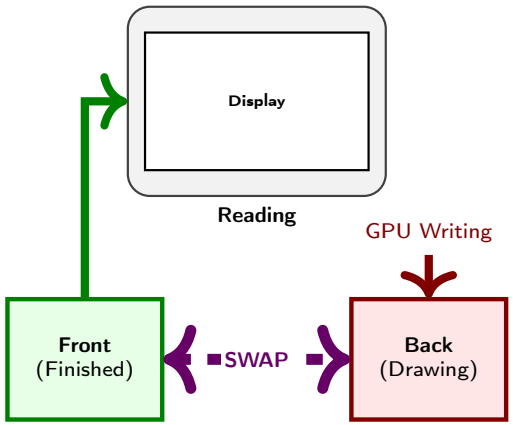


GPU Memory Architecture

Managing Time (Double Buffering) vs. Space (Z-Buffering)

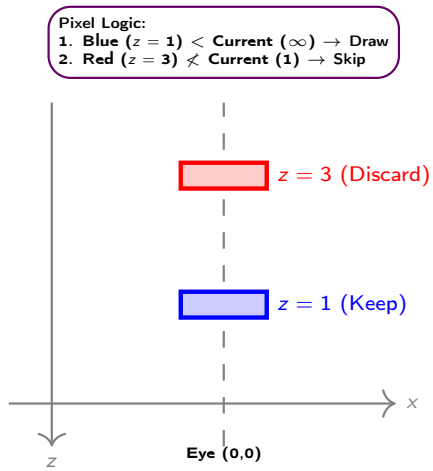
1. Managing TIME (Double Buffering)

- **Goal:** Prevent "Screen Tearing".
- **Concept:** Separate *Writing* from *Reading*.
- **Mechanism: Swap Chain**. The GPU draws to Back; Monitor reads Front. They swap on VSync.



2. Managing SPACE (Z-Buffering)

- **Goal:** Correct Occlusion (Visibility).
- **Concept:** Store Depth (z) per pixel.
- **Mechanism: Depth Test**. Compare fragment z vs. buffer. If $z_{new} < z_{stored}$, keep it.



Cost Analysis: How much VRAM do we need?

To store a frame, we need memory for Colors (Front + Back) and Depth.

$$\text{VRAM} \approx \text{Pixels} \times [(\underbrace{4\text{B}}_{\text{RGBA}} \times \underbrace{2}_{\text{Double Buf.}}) + \underbrace{4\text{B}}_{\text{Depth}}] = \text{Pixels} \times 12 \text{ Bytes}$$

Example (1080p): $1920 \times 1080 \approx 2,000,000$ pixels.

$2\text{M} \times 12 \text{ Bytes} \approx 24 \text{ MB}$ (Minimum for mere display, excluding textures!)

The Rendering Pipeline

The Modern Programmable View (WebGL)

Modern APIs like WebGL abstract these steps into three main conceptual stages, centred on the **programmable** parts:

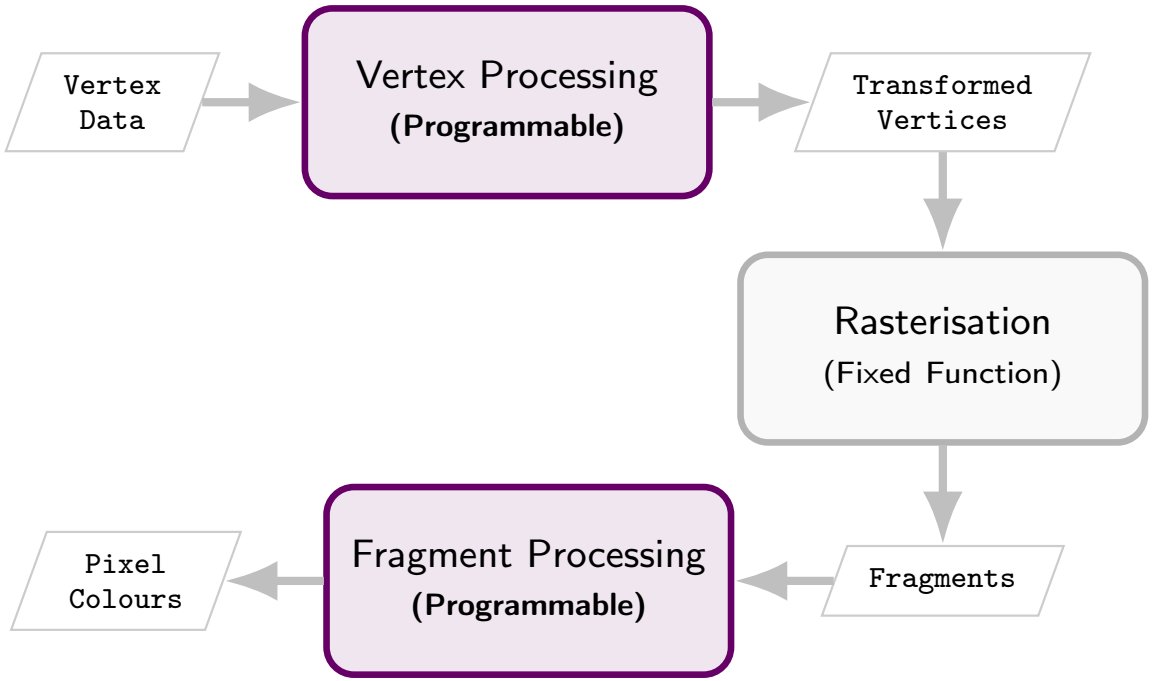
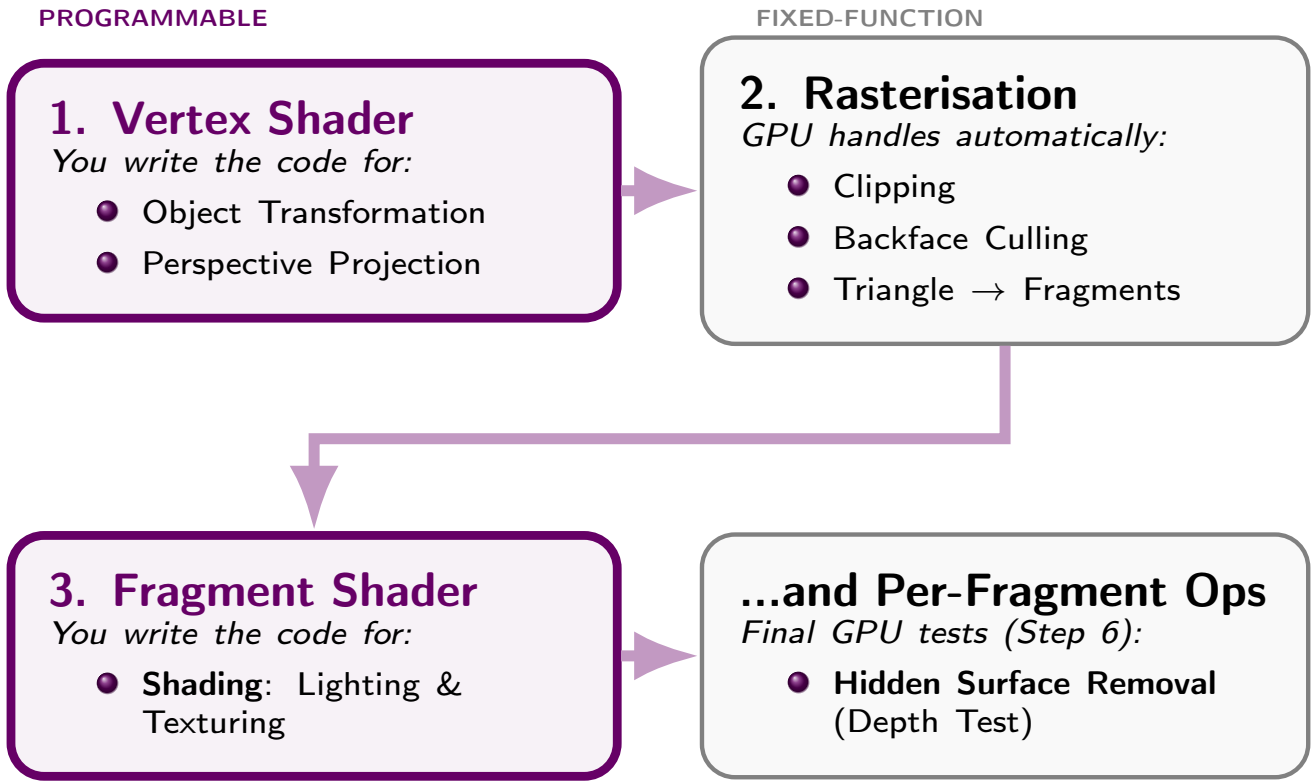


Figure: Data flows through the pipeline stages (Top → Down → Bottom).

Bridging the Two Views

How the Hardware Stages Map to the Shader Model

The "traditional" steps don't disappear; they are grouped into the modern pipeline. The **shaders** give us control over parts that were previously fixed.



- Modern APIs (WebGL) abstract the "factory" into these programmable hooks .

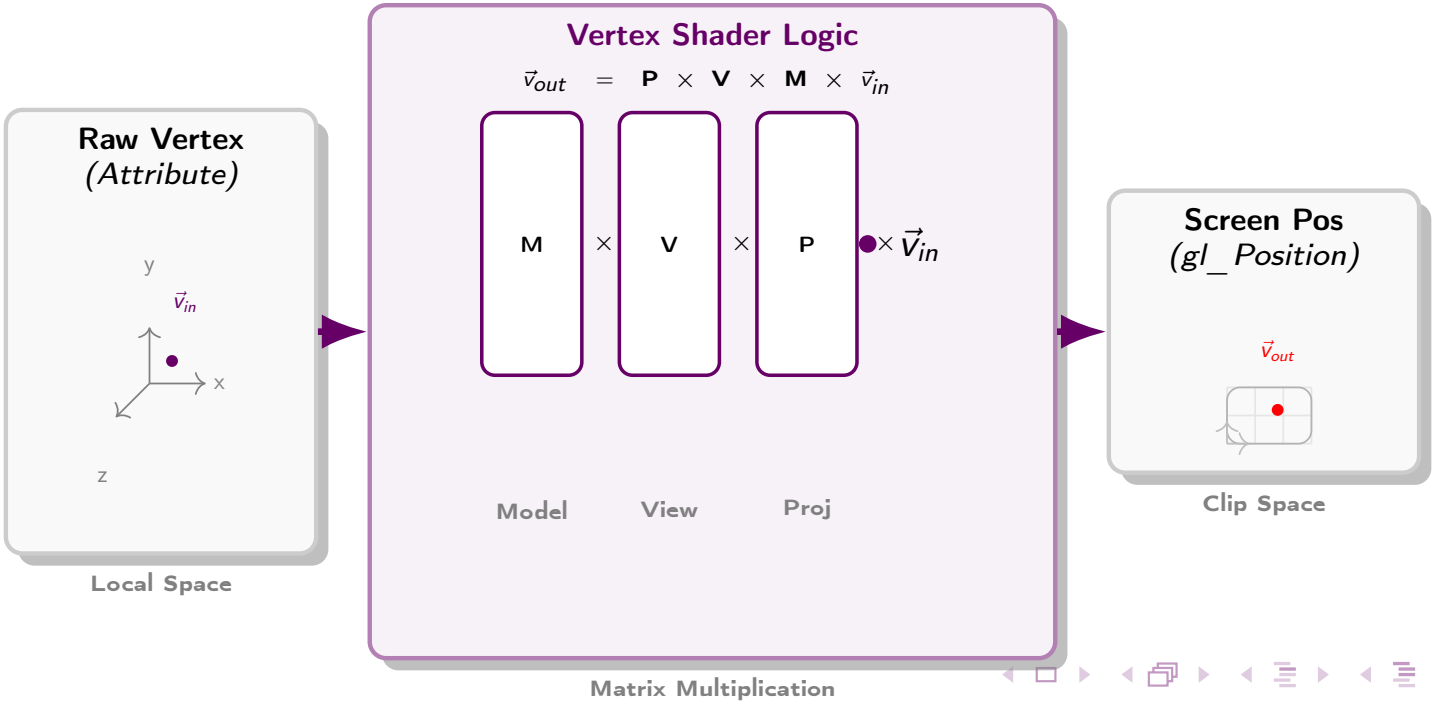
Stage 1: Vertex Processing

The Vertex Shader

This is the first **programmable** stage. The GPU runs a **Vertex Shader** program once for every vertex.

Key Responsibilities

- **Transformation:** Its most critical job is to transform the 3D position of a vertex into the final 2D screen position ("clip space"). This involves a series of matrix multiplications.
- **Data Passthrough:** It processes and passes along other per-vertex data (like texture coordinates or surface normals) to the next stage.



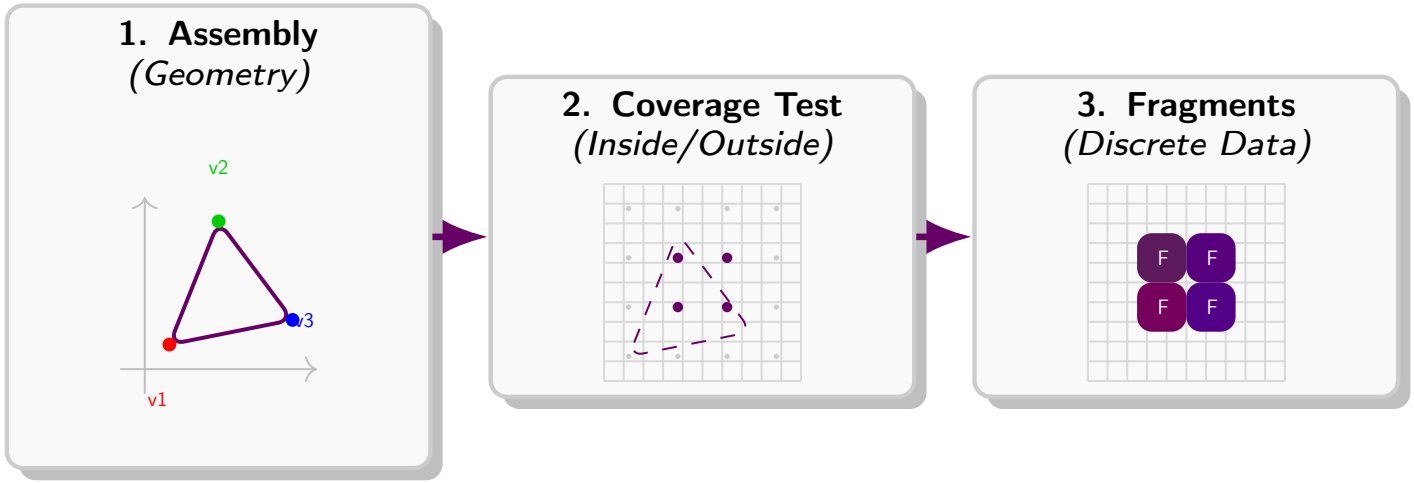
Stage 2: Rasterisation

Fixed Function

This stage is a **fixed, non-programmable** part of the hardware.

Key Responsibilities

- Takes the transformed vertices and connects them to form primitives (usually triangles).
- "Fills in" these triangles, generating a **fragment** (a candidate pixel) for each pixel covered by the triangle.
- Smoothly **interpolates** per-vertex attributes (like colour) across the surface of the triangle for each fragment.



Stage 3: Fragment Processing

The Fragment Shader

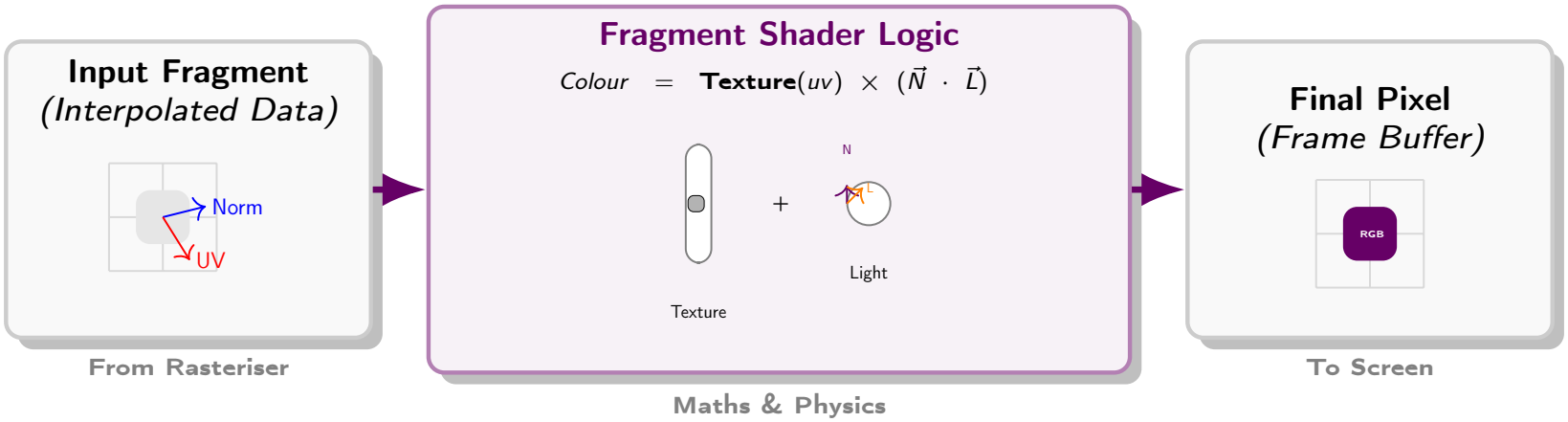
This is the second **programmable** stage. The GPU runs a **Fragment Shader** program once for every fragment.

Key Responsibility

Its sole purpose is to calculate and output the **final colour** for a single fragment.

Common Operations

This is where the visual magic happens: **Texturing** (image lookup) and **Lighting** (physics simulation).

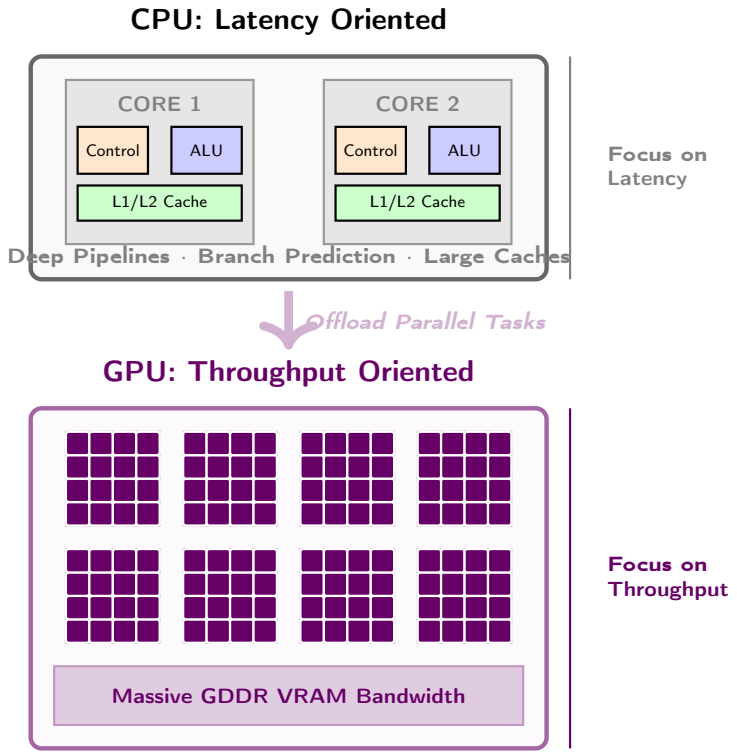


The Power of the GPU

Architecture & Massively Parallel Graphics Processing Unit (GPU)

To achieve the "16-millisecond miracle", the GPU uses a fundamentally different architecture from the CPU. It is built for **throughput** rather than just raw speed per task.

- **Throughput-Oriented Design:**
  - While a CPU is **latency-oriented** (favouring complex control logic for serial tasks), the GPU is **throughput-oriented**, dedicating the majority of its silicon to thousands of simple **ALUs** to process massive data batches simultaneously.
- **Hierarchical Parallelisation (SMs):**
  - Cores are grouped into **Streaming Multiprocessors (SMs)**.
  - Using a **SIMD** (Single Instruction, Multiple Data) model, the GPU executes a single shader program across all ALUs in a group at once, significantly reducing control logic overhead.
- **High-Bandwidth Memory (VRAM):**
  - Optimised with wide data paths to shift vast amounts of vertex, texture, and frame data between memory and the cores instantly, preventing "data starvation" during the 16.6ms rendering window.



Design for Latency (CPU) vs. Throughput (GPU)

OpenGL: The Industry Standard

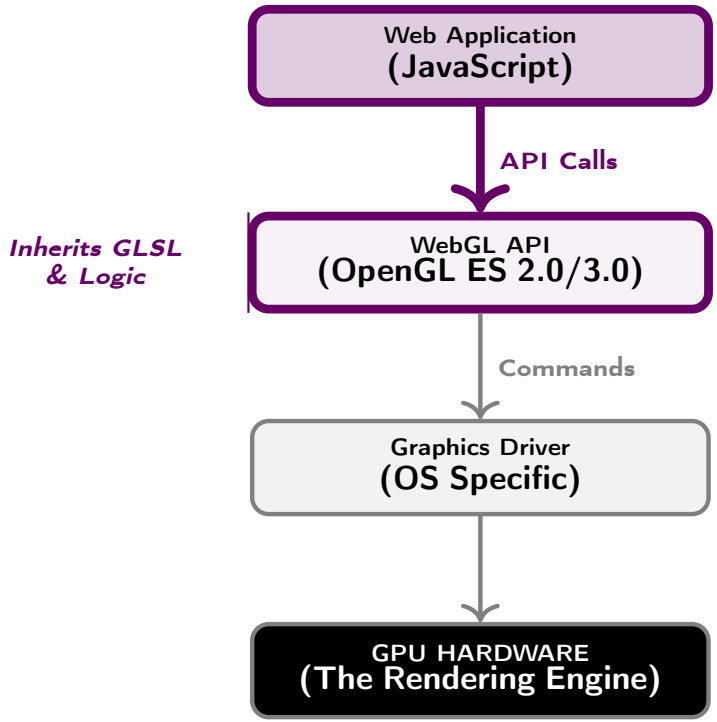
How WebGL Brings GPU Power to the Browser

What is OpenGL?

- **Open Graphics Library:** A cross-language, cross-platform API for instructing the GPU to render 2D and 3D vector graphics.
- **The Foundation:** It provides the *GLSL* (OpenGL Shading Language) used to write the programmable shaders that run on the hardware.

Building WebGL on Top

- **OpenGL ES:** WebGL is based on "OpenGL for Embedded Systems," a streamlined version for mobile and web environments.
- **Web Integration:** It allows browsers to access the GPU directly via JavaScript, providing low-level control over the rendering pipeline.



The Communication Stack

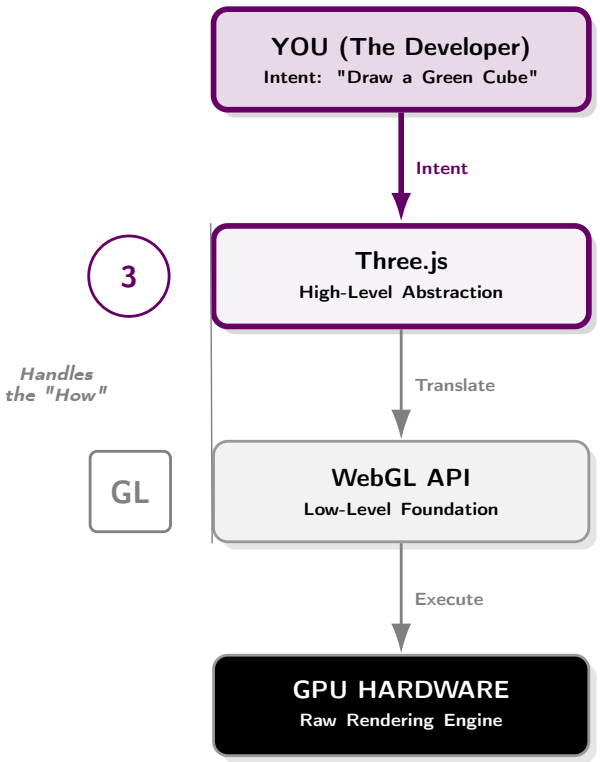
The State Machine

OpenGL/WebGL acts as a **State Machine**: You set the state (textures, shaders, buffers), and the GPU maintains that state until you issue a draw command.

WebGL vs. Three.js: Power vs. Productivity

Choosing the Right Tool for the Job

The Relationship



Comparative Overview

Feature	Raw WebGL	Three.js
Complexity	**Low-Level**: Direct GPU control	**High-Level**: Abstracted API
Concepts	Buffers, Shaders, Attributes	Scene, Camera, Mesh, Light
Code Size	**Verbose**: Large boilerplate	**Concise**: Simple creation
Use Case	Custom engines, raw speed	Rapid development, 3D apps

Summary Perspective

WebGL is the "factory floor" where you manage every machine. **Three.js** is the "Director's Dashboard" that translates your intent automatically.

Practical Implementation with WebGL

WebGL: The HTML Boilerplate

All WebGL graphics are drawn onto a standard HTML <canvas> element.

```
<!DOCTYPE html>
<html>
  <head>
    <title>WebGL Boilerplate</title>
  </head>
  <body>
    <canvas id="glCanvas" width="640" height="480"></canvas>
    <script src="main.js"></script>
  </body>
</html>
```

WebGL: Getting the Rendering Context

The first step is to get the "rendering context" from the canvas. This object is our interface for communicating with the GPU.

```
// Get the canvas element from the HTML document
const canvas = document.getElementById('glCanvas');

// Get the WebGL rendering context
const gl = canvas.getContext('webgl');

if (!gl) {
  alert('Unable to initialize WebGL.');
}
```

Analysis

`canvas.getContext('webgl')` requests a WebGL context. The resulting `gl` object holds all the WebGL functions we will use.

WebGL: A Basic Vertex Shader

Shaders are written in **GLSL** (OpenGL Shading Language). The vertex shader's minimum job is to set the special `gl_Position` variable.

```
// An 'attribute' is an input to the vertex shader.
// It receives data from a GPU buffer we set up.
attribute vec2 a_position;
void main() {
  // gl_Position is a special, required output variable.
// It determines the final position of the vertex.
  // WebGL requires a 4D vector (x, y, z, w).
gl_Position = vec4(a_position, 0.0, 1.0);
}
```

WebGL: A Basic Fragment Shader

The fragment shader's minimum job is to set the special `gl_FragColor` variable, which defines the final colour of a pixel.

```
// This sets the default precision for floating point numbers.
// It's a mandatory line in WebGL fragment shaders.
precision mediump float;

void main() {
  // gl_FragColor is a special, required output variable.
// It expects a 4D vector (Red, Green, Blue, Alpha).
  gl_FragColor = vec4(1.0, 0.0, 0.0, 1.0);
// Solid, opaque Red
}
```

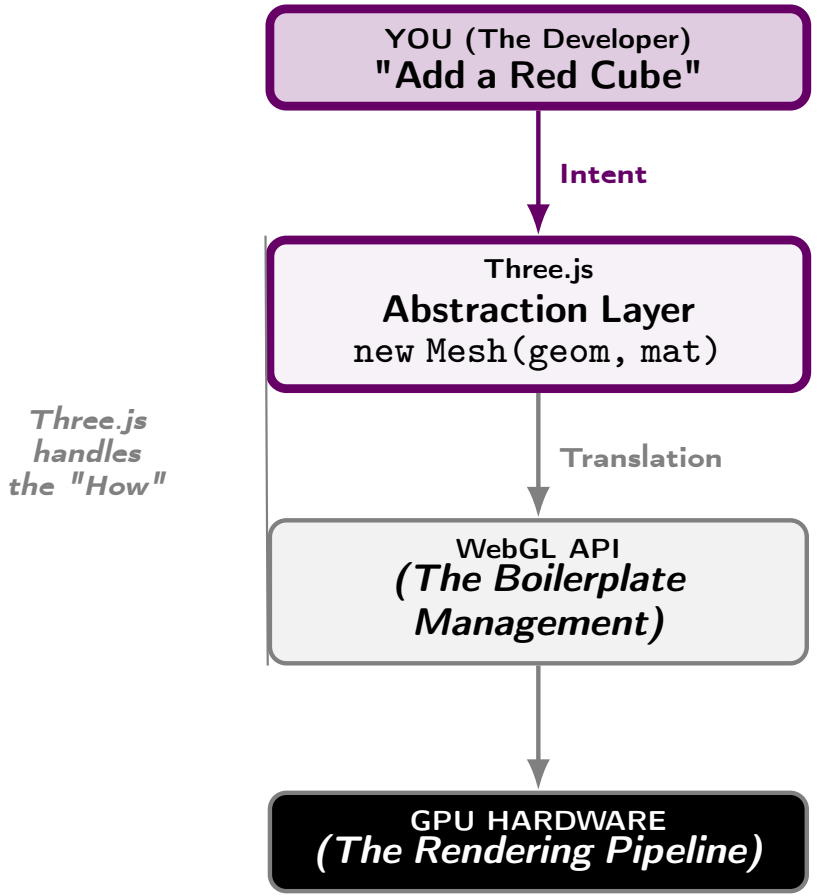
Three.js: The Director's Dashboard

Elevating Creative Control over Technical Complexity

The Challenge: WebGL Verbosity
WebGL is a powerful "raw" interface to the GPU, but it is notoriously verbose and low-level. Setting up a single triangle can require dozens of lines of boilerplate code.

The Solution: Three.js
Three.js is a JavaScript library that provides a **higher level of abstraction**. It acts as a bridge, sitting on top of WebGL to handle the technical "heavy lifting".

- **Intuitive Objects:** Manipulate the world via `Scene`, `Camera`, and `Light` rather than buffers and shaders.
- **Creative Focus:** Shift your energy toward **composition and interaction** rather than implementation details.



The Result

Productivity: Build complex 3D environments in minutes that would take days in raw WebGL.

Anatomy of a Three.js Program

The "Big Three" Architecture: Stage, Actors, and Action

PHASE 1: SETUP (Runs 1x)

1. The Stage (Boilerplate)

```
// 1. The World (Scene)
const scene = new THREE.Scene();

// 2. The Eye (Camera)
const cam = new PerspectiveCamera(...);
cam.position.z = 5;

// 3. The Painter (Renderer)
const ren = new WebGLRenderer();
document.body.append(ren.domElement);
```

2. The Actors (Content)

```
// Geometry (Shape)
const geo = new BoxGeometry(1,1,1);

// Material (Skin/Shader)
const mat = new MeshBasicMaterial({
  color: 0x00ff00
});

// Mesh (The Object)
const cube = new Mesh(geo, mat);

// ENTER STAGE
scene.add(cube);
```

Data Flow

PHASE 2: LOOP (Runs 60Hz)

3. The Action (Render Loop)

```
function animate() {
  requestAnimationFrame(animate);

  // CPU Logic (Update)
  cube.rotation.x += 0.01;
  cube.rotation.y += 0.01;

  // GPU Paint (Draw)
  ren.render(scene, cam);
}

animate(); // Start Loop
```

*The **Scene Graph** (Left) defines structure; the **Render Loop** (Right) creates time.*

Three.js Example: A Spinning Cube

Part 1: Setup

```
<!DOCTYPE html>
<html>
<head><title>Three.js Spinning Cube</title></head>
<body>
<script type="importmap">
  { "imports": { "three":
  "https://cdn.jsdelivr.net/npm/three@0.160.0/build/three.module.js" } }
</script>
<script type="module">
  import * as THREE from 'three';
// 1. Scene Setup
  const scene = new THREE.Scene();
const camera = new THREE.PerspectiveCamera(75,
      window.innerWidth / window.innerHeight, 0.1, 1000);
const renderer = new THREE.WebGLRenderer();
  renderer.setSize(window.innerWidth, window.innerHeight);
  document.body.appendChild(renderer.domElement);
  // ... code continues on next slide
</script>
</body>
</html>
```

Three.js Example: A Spinning Cube

Part 2: Object & Animation

```
  // ... continued from previous slide

  // 2. Object Creation
  const geometry = new THREE.BoxGeometry(1, 1, 1);
const material = new THREE.MeshBasicMaterial({ color: 0x00ff00 }); // Green
  const cube = new THREE.Mesh(geometry, material);
  scene.add(cube);
camera.position.z = 5;

  // 3. Render Loop
  function animate() {
    requestAnimationFrame(animate);
    cube.rotation.x += 0.01;
cube.rotation.y += 0.01;
    renderer.render(scene, camera);
  }
  animate();
</script>
```

Analysis of the Three.js Code

1. **Scene Setup:**
   - `Scene`: A container for all objects, lights, and cameras.
   - `Camera`: Defines our viewpoint into the scene.
   - `Renderer`: The workhorse that renders the scene from the camera's perspective onto a `<canvas>`.
2. **Object Creation:**
   - `Geometry`: Defines the shape of an object (e.g., a cube's vertices).
   - `Material`: Defines the appearance of an object's surface (e.g., a solid green colour).
   - `Mesh`: The final object that combines a geometry with a material.
3. **Render Loop:**
   - A function that runs every frame to draw the scene.
   - We update the cube's `rotation` here to create animation.
   - `renderer.render(scene, camera)` is the key command that executes the entire rendering pipeline for us.

Lecture 1: Wrap-up

Key Takeaways

- **Computer Graphics** blends art and science to turn 3D data into images via Modelling, Rendering, and Animation.
- We focus on **Rasterisation**, the dominant technique for real-time graphics.
- The **Hardware Pipeline** consists of specific logical steps: Transformation $\rightarrow$ Projection $\rightarrow$ Clipping $\rightarrow$ Backface Culling $\rightarrow$ Rasterisation $\rightarrow$ Hidden Surface Removal and Shading.
- Modern **Shaders** replace fixed steps with programmable code:
  - **Vertex Shaders** handle Transformation and Projection.
  - **Fragment Shaders** handle Shading and colour calculations.
- Critical steps like **Clipping**, **Backface Culling**, and the **Depth Test** remain optimised fixed-function hardware operations.
- **WebGL** gives low-level control over this pipeline, while **Three.js** provides a high-level, intuitive API.

Questions?

The 16-Millisecond Miracle: Summary

Computer Graphics is the powerful blend of physics, mathematics, and art used to transform abstract descriptions into compelling images. We teach "silicon rocks" to perform a magic trick: turning boring coordinates into millions of pixels to deceive the human brain into seeing depth.

1. THE ARCHITECTURE
Use Linear Algebra to move from Local to World space. Shaders give you programmable control over the factory.

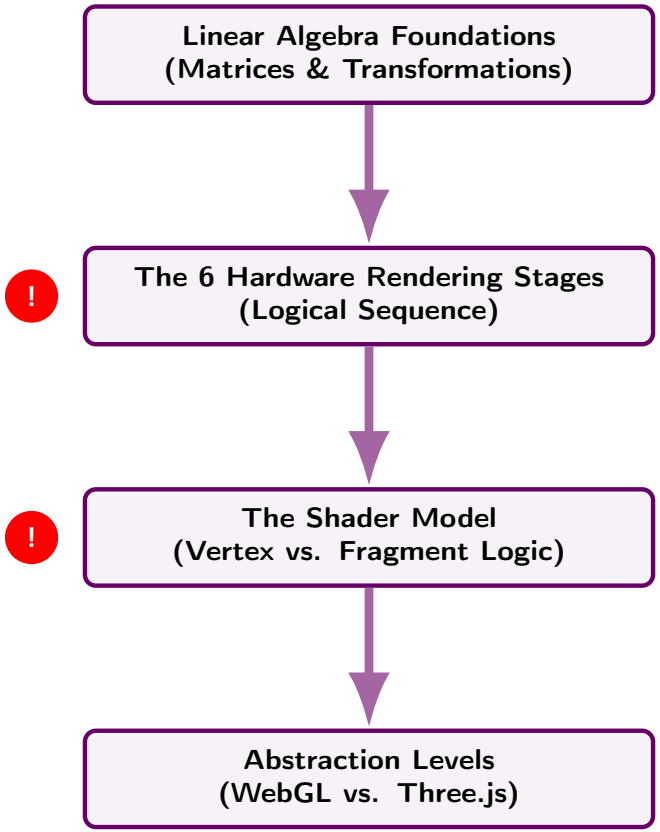
2. THE LOGISTICS
Rasterisation is the bridge between math and pixels. GPUs use SIMD to process millions of triangles in a blink.

3. THE INTERFACE
Three.js elevates creative control, handling the WebGL boilerplate so you can focus on composition.

Study Guide for This Lecture

Focus Areas and Learning Strategies

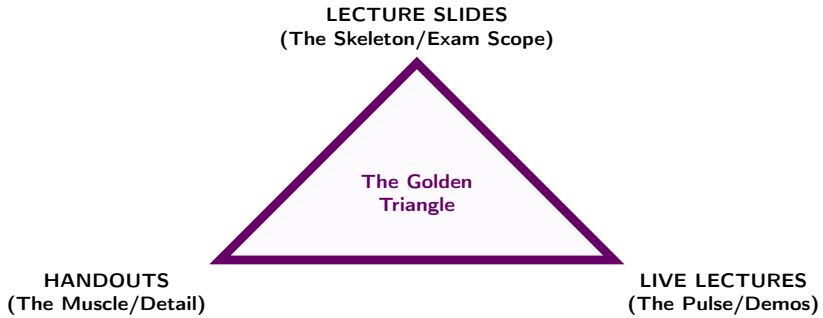
1. The Core Knowledge Path



Priority 1: The Functional Mapping

Always trace the data: Transformation happens in the Vertex Shader while Shading is calculated in the Fragment Shader.

2. Learning Strategies

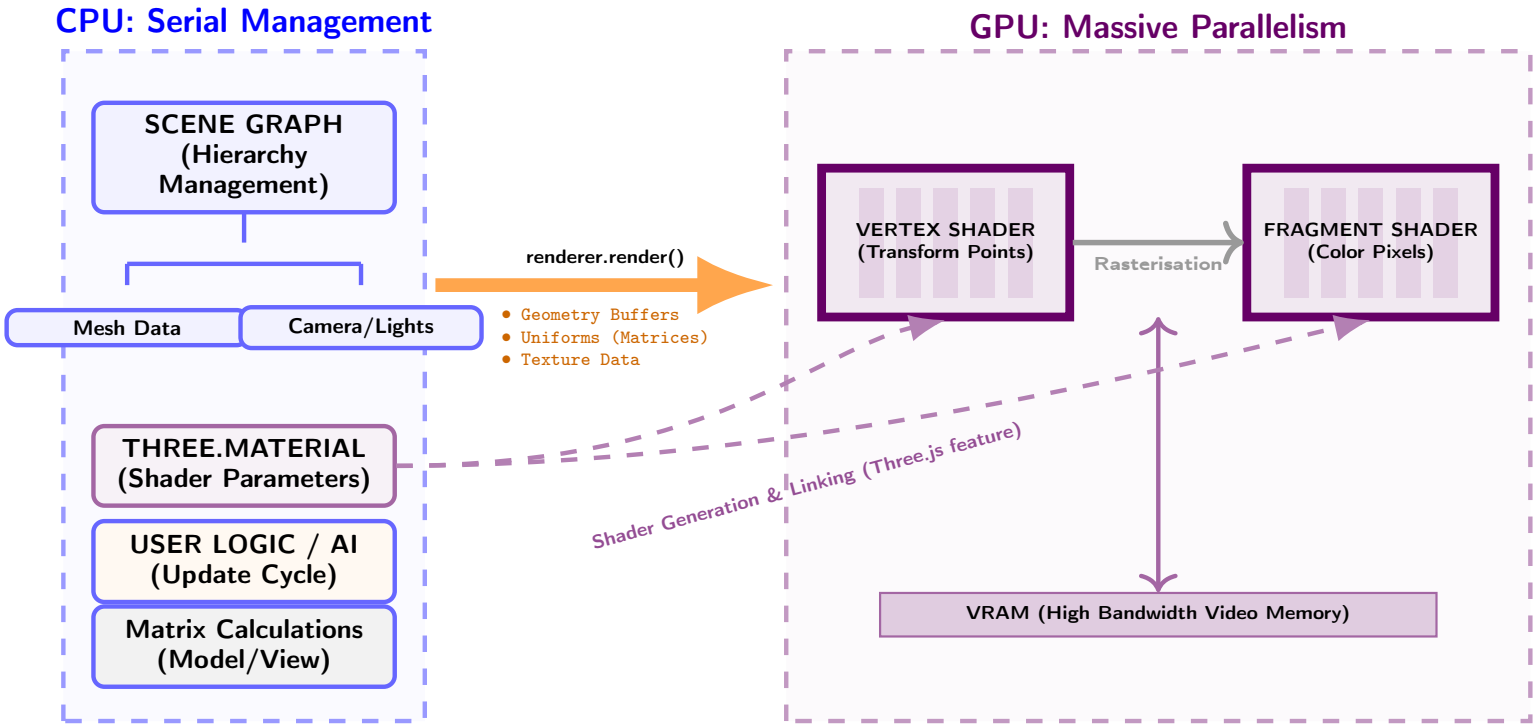


Critical Study Habits

- No Shortcuts: **The pipeline is sequential; missing one stage breaks the logic of the next.**
- Data Flow: **Focus on how *numbers (i.e., geometric data)* become *pixels*.**
- Active Coding: **Experiment with Three.js to see abstraction in action.**

Appendix: Anatomy of a Three.js Program

How Materials Bridge CPU Logic to GPU Shaders



- **Three.js Material Factory**: Materials (e.g., `MeshPhongMaterial`) act as high-level templates that stitch together GLSL snippets into final shaders.
- **Direct Linkage**: The CPU manages the material state, but the actual mathematical execution (lighting/textures) is offloaded to the GPU's parallel cores.

Appendix: Teapot Program as an Example

Distinguishing CPU Management from GPU Execution

CPU: The Orchestrator (JavaScript)

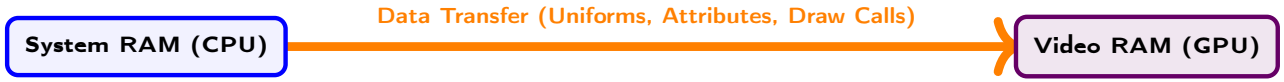
- Setup & State: **Builds the scene graph and passes custom GLSL code to the** `ShaderMaterial`.
- Uniform Management: **Defines and updates constant values (like** `uColor` **and** `uLightPos`**) sent to the GPU hardware.**
- The Call: **The** `animate()` **loop uses** `renderer.render()` **to trigger the GPU handoff 60 times a second.**

```
// 1. CPU sets the "Uniform" state & Shaders
const material = new THREE.ShaderMaterial({
  vertexShader: _VS, fragmentShader: _FS,
  uniforms: { uColor: { ... }, uLightPos: { ... } }
});

// 2. CPU defines the Object (Mesh)
const geometry = new TeapotGeometry(1, 10);
const teapot = new THREE.Mesh(geometry, material);
scene.add(teapot);

// 3. CPU Loop: Trigger GPU Draw Call
function animate() {
  requestAnimationFrame(animate);
  renderer.render(scene, camera);
}
```

GPU: The Worker (GLSL)

- Vertex Processing: **The Vertex Shader transforms 3D points to 2D screen space.**
- Fragment Processing: **The Fragment Shader calculates the color of each pixel.**
- Interpolation: **Automatically blends the** `varying` **values (**`vNormal`, `vViewPos`**) between stages.**

```
/* VERTEX SHADER (Runs for each point) */
void main() {
  vNormal = normalize(normalMatrix * normal);
  vec4 viewPosition = modelViewMatrix * vec4(position, 1.0);
  vViewPos = -viewPosition.xyz;
  gl_Position = projectionMatrix * viewPosition;
}

/* FRAGMENT SHADER (Runs for each pixel) */
void main() {
  vec3 lightDir = normalize(uLightPos - vViewPos);
  float diff = max(dot(normalize(vNormal), lightDir), 0.0);
  gl_FragColor = vec4(uColor * (diff + 0.2), 1.0);
}
```



Appendix: Code Volume Comparison

The "Triangle Test": ∼70 Lines vs. ∼15 Lines (WebGL vs. Three.js)

WebGL: The "Manual Factory"

Rendering a triangle requires manual management of the entire GPU state:

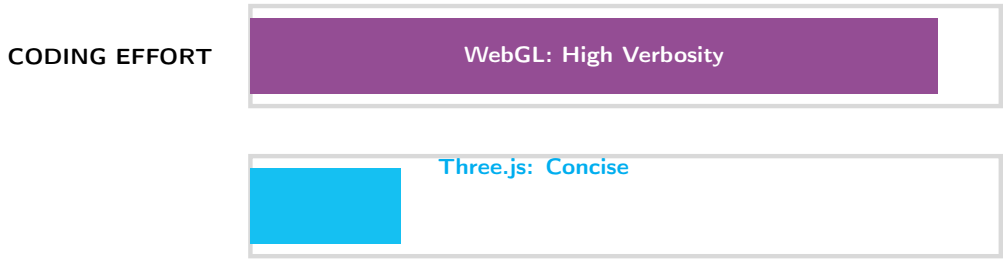
1. Fetch GL context.
2. Write/Compile Vertex Shader.
3. Write/Compile Fragment Shader.
4. Link into a Shader Program.
5. Define Float32Array vertices.
6. Create, Bind, and Buffer GPU data.
7. Map attributes to buffers.
8. Issue the `drawArrays` call.

Three.js: The "Automated Line"

Three.js encapsulates these steps into high-level objects:

1. Initialise `Renderer` `Scene`.
2. Create `BufferGeometry`.
3. Create `Material`.
4. Combine into a `Mesh`.
5. Add to scene and `render()`.

Boilerplate like shader compilation and attribute mapping is handled automatically.



Appendix: Code Volume Comparison

The "Triangle Test": Imperative vs. Declarative

Raw WebGL (Imperative)

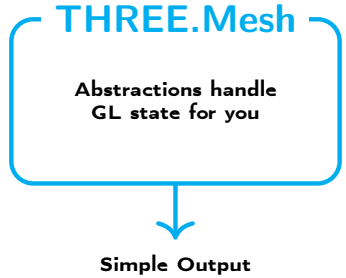
```
// 1. Compile Shaders (Manual!)
const vS = gl.createShader(gl.VERTEX_SHADER);
gl.shaderSource(vS, `attr vec2 p;void main(){gl_Position=
      vec4(p,0,1);}`);
gl.compileShader(vS);
const fS = gl.createShader(gl.FRAGMENT_SHADER);
gl.shaderSource(fS, `void main(){gl_FragColor=vec4
      (1,0,0,1);}`);
gl.compileShader(fS);

// 2. Link Program
const prog = gl.createProgram();
gl.attachShader(prog, vS);
gl.attachShader(prog, fS);
gl.linkProgram(prog);
gl.useProgram(prog);

// 3. Buffer Data (Manual Memory!)
const vbo = gl.createBuffer();
gl.bindBuffer(gl.ARRAY_BUFFER, vbo);
gl.bufferData(gl.ARRAY_BUFFER, new Float32Array
      ([0,1,-1,-1,1,-1]), gl.STATIC_DRAW);

// 4. Attribute Pointers (The Handshake)
const loc = gl.getAttribLocation(prog, "p");
gl.enableVertexAttribArray(loc);
gl.vertexAttribPointer(loc, 2, gl.FLOAT, false, 0, 0);

// 5. Draw
gl.drawArrays(gl.TRIANGLES, 0, 3);
```

The Burden

You manage memory, compilation, and state manually.

Three.js (Declarative)

```
// 1. Geometry (Data)
const geom = new THREE.BufferGeometry();
const vertices = new Float32Array([
  0,1,0, -1,-1,0, 1,-1,0
]);
geom.setAttribute('position',
  new THREE.BufferAttribute(vertices, 3));

// 2. Material (State)
// Shaders compiled automatically!
const mat = new THREE.MeshBasicMaterial({
  color: 0xff0000
});

// 3. Mesh & Render
const mesh = new THREE.Mesh(geom, mat);
scene.add(mesh);
renderer.render(scene, camera);
```



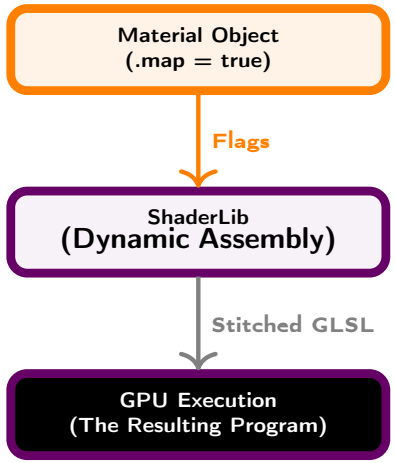
Appendix: From Material to Shader in Three.js

How Material Properties Generate GLSL

Three.js acts as a **Shader Factory**. It maintains a library of pre-written snippets (ShaderChunks) and "stitches" them together based on your material settings.

The Generation Process: When you set `material.map`, Three.js adds a `#define USE_MAP` to the top of its "Uber-Shader" template.

```
// 1. User sets Intent (High-Level)
const material = new THREE.MeshPhongMaterial({
    color: 0xff0000,
    map: myTexture // Triggers USE_MAP
});

// 2. Three.js Internal Generation (Stitching)
// The engine produces a GLSL string like this:
const generatedShader = `
    #define USE_MAP
    #define USE_LIGHTS

    uniform sampler2D map;
    varying vec2 vUv;

    void main() {
        vec4 color = vec4(1.0, 0.0, 0.0, 1.0);
        #ifdef USE_MAP
            color *= texture2D(map, vUv); // Stitched in!
        #endif
        gl_FragColor = color;
    }
`;
```



Recommendation to Students

While Three.js automates standard lighting (Point/Directional), you must study shader programming because:

- Design Flexibility: **Standard materials cannot create custom effects like liquid ripples or holographic distortions.**
- Optimisation: **Auto-generated "Uber-Shaders" can be inefficient; custom shaders are leaner for mobile/web.**
- Pipeline Mastery: **You cannot debug complex rendering artifacts without understanding the math inside the "Black Box".**

Appendix: What to pay attention

What to pay attention to in each environment (WebGL vs. Three.js)

WebGL: State Management

Pay Attention To:

- **Global State:** WebGL is a state machine; failing to "unbind" a buffer can cause unexpected side effects in later draw calls.
- **Type Safety:** Data passed to the GPU (Float32Arrays) must be perfectly aligned with shader attributes.
- **Error Handling:** Shaders fail silently; you must manually check compilation logs.

Three.js: Abstraction Costs

Pay Attention To:

- **The "Black Box":** Because Three.js handles the "How", debugging raw performance issues requires understanding what it does under the hood.
- **Object Lifecycle:** Meshes must be explicitly disposed of to prevent memory leaks in the browser.
- **Hidden Overhead:** Every `Mesh` and `Material` adds management overhead; 10,000 individual meshes will be slower than one custom WebGL buffer.

The Decision Metric

Choose WebGL when you need a custom engine, highly specialised data structures, or maximum raw performance optimisation.
Choose Three.js for 95% of general-purpose 3D web applications, interaction, and creative storytelling where productivity is paramount.